

PRÁCTICA 1

CONEXIA: Plataforma de conexión y optimización de redes.

Programación orientada a objetos

**Grupo 01
Equipo 10**

**Maria Jose Monroy Mejia
Valeria Moreno Rojas
Justin Camilo Loaiza Lujan**

Jaime Alberto Guzmán Luna

Universidad Nacional de Colombia

Sede Medellín

2025

Índice

1. Portada	1
2. Descripción general de la solución	4
2.1. Identificación y requisitos funcionales del problema	4
2.2. Enfoque del sistema	4
2.3. Diseño del sistema	4
3. Descripción del diseño estático del sistema en la especificación UML	6
3.1. Cliente	6
3.2. ProveedorInternet	7
3.3. Plano	8
3.4. Plan	8
3.5. Factura	9
3.6. Servidor	9
3.7. Router	10
3.8. Dispositivo	10
3.9. Antena	11
3.10. Cobertura (Abstracta)	11
3.11. Red (Interfaz)	12
4. Descripción de la implementación de características de programación orientada a objetos en el proyecto	13
4.1. Clase abstracta y método abstracto	13
4.2. Interfaz (método default y método estático)	14
4.3. Herencia	15
4.4. Ligadura dinámica (2 casos)	15
4.5. Ligadura estática	16
4.6. Atributos de clase y métodos de clase	17
4.7. Uso de constante	18
4.8. Encapsulamiento (Private, protected y public)	18
4.9. Sobrecarga (1 de métodos y 2 de constructores)	18
4.10. Referencias this y this()	21
4.11. Implementación de un caso de enumeración	22
5. Descripción de cada una de las 5 funcionalidades implementadas	23
5.1. Funcionalidad 1: Adquisición de un plan	24
5.2. Funcionalidad 2: Visualizar los dispositivos conectados	27
5.3. Funcionalidad 3: Mejora tu plan	29
5.4. Funcionalidad 4: Test	33
5.5. Funcionalidad 5: Reporte de fallas en el servidor	37
6. Manual de usuario	41
6.1. Seleccionar la opción 1	41
6.2. Seleccionar la opción 2	44

6.2.1 Pagos al día	44
6.2.1 Pagos pendientes	45
6.3. Seleccionar la opción 3	47
6.4. Seleccionar la opción 4	48
6.5. Seleccionar la opción 5	49
6.6. Seleccionar la opción 6	51

2. Descripción general de la solución.

2.1. Identificación y requisitos funcionales del problema.

El problema que se busca solucionar es la falta de rapidez en la respuesta al usuario, ya que procesos como cambiar de plan, adquirir un router o reportar problemas con el servicio pueden ser muy demorados debido a la gestión administrativa. Esta demora afecta la experiencia del usuario y la eficiencia operativa de la empresa. Para abordar esta problemática, se está desarrollando una aplicación que permite a los usuarios realizar múltiples acciones en un solo lugar, ahorrando tiempo y mejorando su experiencia con nuestra empresa. De esta manera, la aplicación facilita la modificación de servicios, la adquisición de planes, la mejora de la conectividad y la resolución de incidencias de forma más ágil y eficiente.

2.2. Enfoque del sistema.

El dominio del proyecto está centrado en una empresa llamada Conexia, dedicada a la distribución y modificación de planes de Internet. La solución tiene como objetivo principal facilitar la modificación y adquisición de planes de Internet, mediante la implementación de cinco funcionalidades principales:

1. Adquisición de un plan: Permite a los usuarios seleccionar un proveedor y un plan de servicio (Basic, Standard o Premium).
2. Visualización de dispositivos conectados: Muestra los dispositivos que están conectados a la red del usuario.
3. Mejora de un plan: Ofrece recomendaciones para mejorar el plan de acuerdo con el uso actual del servicio.
4. Prueba de velocidad de Internet (test): Realiza una prueba de velocidad de conexión y proporciona recomendaciones.
5. Informe o reporte de anomalías: Genera un informe sobre posibles problemas en los servidores de los proveedores.

El sistema está diseñado para ejecutarse en un entorno de usuario único, lo que significa que todas las funcionalidades están dirigidas a mejorar la experiencia de un solo usuario, que interactúa con la consola desde diferentes maneras y propósitos.

2.3. Diseño del sistema.

El diseño del sistema de redes está modelado en un plano cartesiano de 100x100, que representa una abstracción de la realidad. Este espacio incluye cuatro localidades principales, denominadas A, B, C y D, donde se distribuyen componentes clave como antenas, servidores y routers. Estos elementos interactúan de manera lógica para soportar las funcionalidades de la aplicación.

Las antenas tienen zonas de cobertura representadas como circunferencias, cuyo radio varía según la generación de tecnología utilizada:

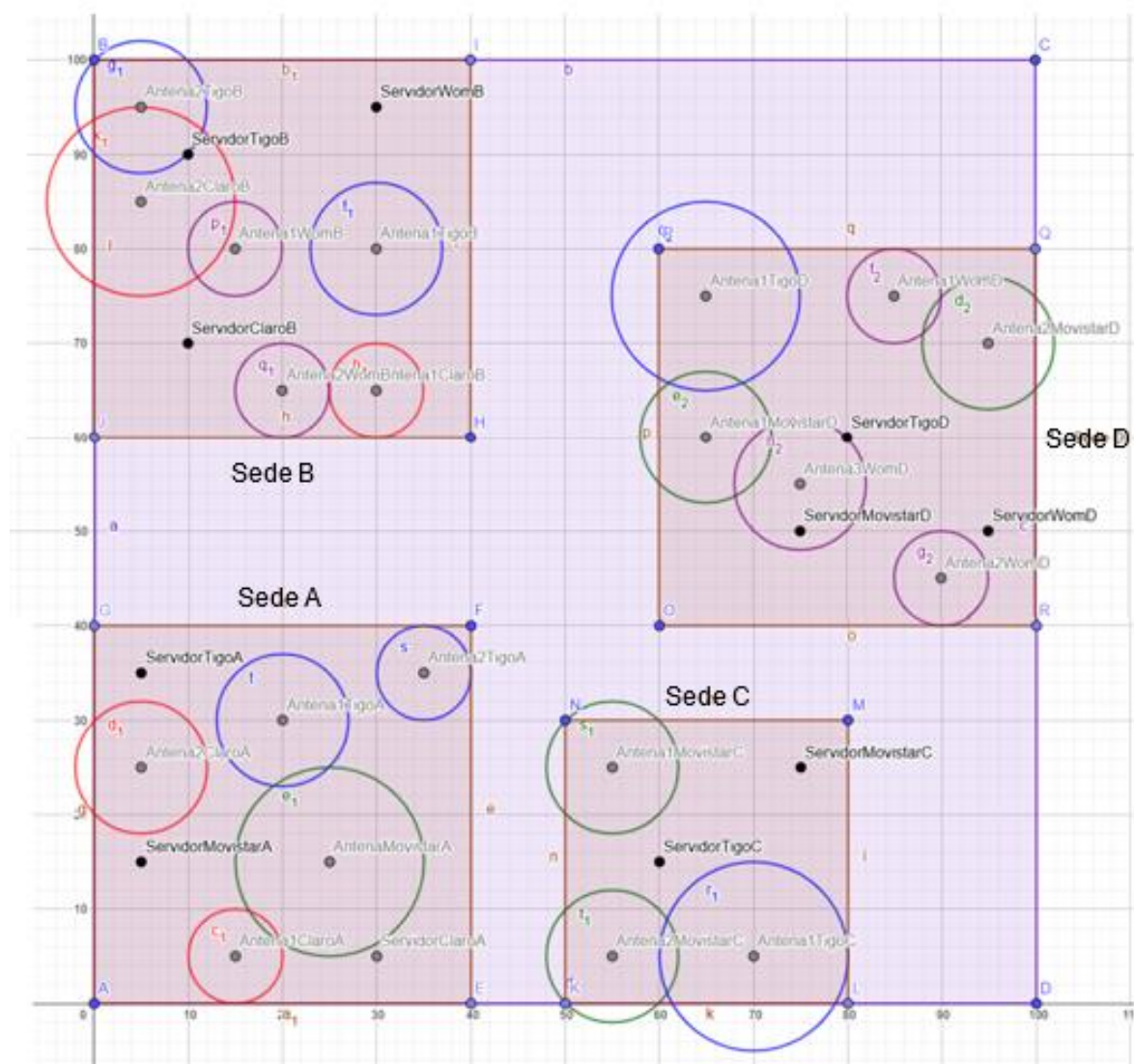
- 3G: radio de 5 unidades.
- 4G: radio de 7 unidades.
- 5G: radio de 10 unidades.

Además, los proveedores de servicio están identificados mediante colores específicos en el esquema:

- Azul para Tigo.
- Verde para Movistar.
- Rojo para Claro.
- Morado para Wom.

Este diseño permite a los usuarios realizar mejoras en su servicio, visualizar los dispositivos conectados a su red y gestionar sus planes de manera más eficiente y rápida, optimizando los tiempos y mejorando significativamente la experiencia general.

A continuación, se presenta un esquema que ilustra la distribución de las antenas, routers y zonas de cobertura dentro del sistema. Este gráfico muestra cómo se organizan los elementos clave en el plano cartesiano.



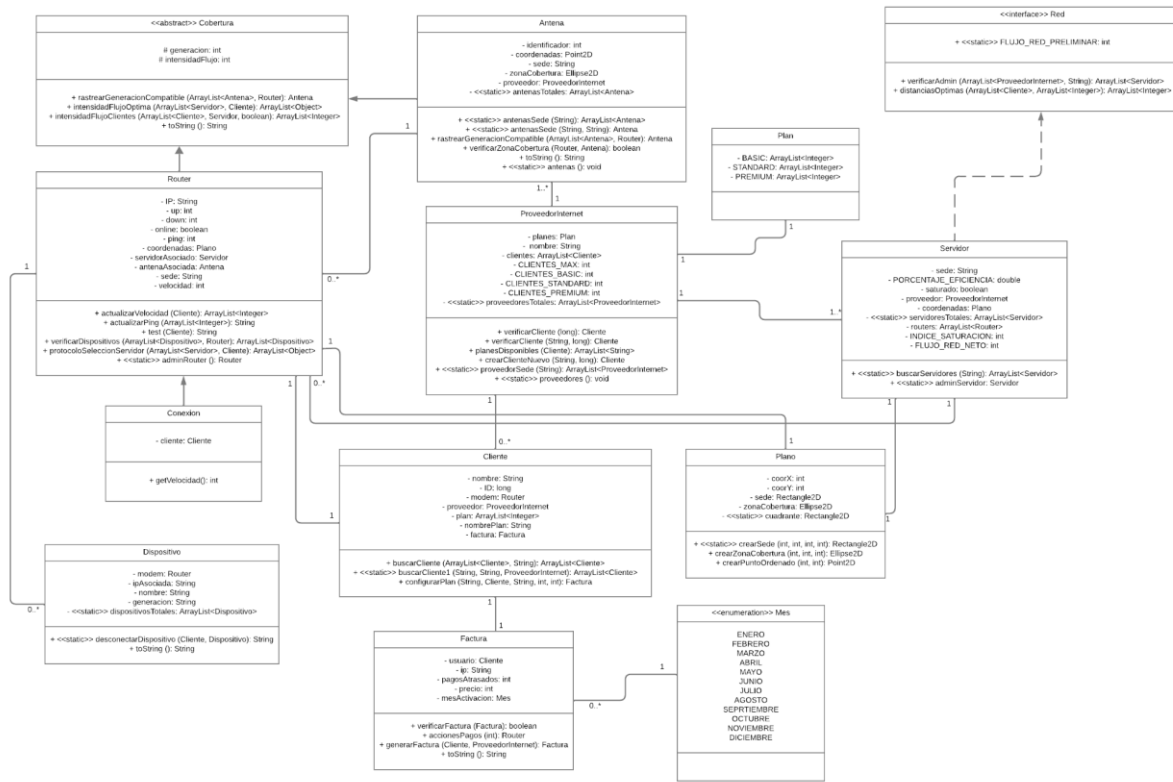
En caso de necesitar mayor detalle, se puede consultar el siguiente enlace:

<https://www.geogebra.org/classic/kmd7rzk5>

3. Descripción del diseño estático del sistema en la especificación UML.

CONEXIA

Plataforma de conexión y optimización de redes.



El diagrama de UML que esboza las clases del sistema y la relación que existe entre ellas. Así mismo se anexa el enlace que permite visualizar dicho diagrama de mejor manera. Enlace:

https://lucid.app/lucidchart/e66db1d9-fbf5-476d-ac92-7bc31a33922c/edit?viewport_loc=-947%2C-197%2C5810%2C2170%2CHWEp-vi-RSFO&invitationId=inv_f276f07a-ad2a-4508-ace9-f9e33a19677f

A continuación, se describe cada una de las clases implementadas:

3.1. Cliente.

La clase Cliente representa a los usuarios que contratan el servicio a internet. Esta clase es responsable de manejar información sobre los clientes, como sus datos personales, el plan de internet contratado, el proveedor asociado, y los dispositivos conectados. Además, incluye métodos para buscar clientes, asignar planes de servicio y generar facturas.

Métodos Principales:

- buscarCliente (Instancia): Devuelve una lista de clientes que tienen un plan específico.
- buscarCliente1 (Estático): Busca clientes que pertenezcan a un mismo proveedor, sede y plan.
- configurarPlan (Instancia): Realiza todos los cambios necesarios para registrar un cliente en un nuevo plan de servicio. Esto incluye asignar un servidor, un router, y una antena del proveedor asociado. Retorna: Una nueva factura (Factura) generada

para el cliente.

- Los Getters y Setters: Son para acceder y modificar los atributos privados de la clase.

Relación con Otras Clases:

- Router: Representa el modem asociado al cliente.
- ProveedorInternet: Define el proveedor del servicio de internet.
- Factura: Registra los costos y detalles del plan contratado.
- Antena y Servidor: Son utilizados en la configuración de los planes y enrutamiento de la conexión.

3.2. ProveedorInternet.

La clase ProveedorInternet los proveedores de servicios de internet disponibles en el sistema. Esta clase se encarga de manejar información relacionada con los planes de servicio ofrecidos, los clientes asociados y la capacidad de administración de recursos. Además, incluye funcionalidades para verificar clientes, consultar planes disponibles y gestionar la asignación de clientes.

Métodos Principales:

- verificarCliente (Instancia): Verifica si un cliente con un ID específico está asociado al proveedor y tiene un plan activo (BASIC, STANDARD o PREMIUM). También permite comprobar clientes por nombre e ID, validando que tengan más de dos dispositivos conectados.
- planesDisponibles (Instancia): Determina qué planes están disponibles para un cliente, basándose en los cupos restantes en cada plan.
- crearClienteNuevo (Instancia): Permite crear un nuevo cliente asociado al proveedor con el nombre e ID especificados.
- proveedorSede (Estático): Encuentra todos los proveedores que operan en una sede específica, utilizando los servidores registrados.
- proveedores (Estático): Actualiza la lista de proveedores totales extrayendo los proveedores registrados en los servidores.
- Los Getters y Setters: Métodos para acceder y modificar los atributos privados de la clase.

Relación con Otras Clases:

- Con Cliente: Paraverificar clientes (verificarCliente), crear nuevos clientes (crearClienteNuevo) y consultar qué clientes están vinculados a un plan específico.
- Con Plan: Cada proveedor gestiona los planes de servicio (BASIC, STANDARD, PREMIUM) que ofrece a los clientes.
- Con Servidor: Los servidores están asociados a un ProveedorInternet y distribuyen los recursos de red a los clientes del proveedor.
- Con Dispositivo: los dispositivos conectados a un cliente están vinculados al proveedor a través del router y los planes asociados.
- Con Router: Los routers asociados a los clientes y estos a proveedor y los planes contratados.
- Con Antena: Las antenas asociadas al proveedor aseguran la cobertura de red en áreas específicas.
- Con Factura: Las facturas generadas para los clientes del proveedor dependen de los planes contratados y los recursos utilizados.

3.3. Plano.

La clase Plano define las coordenadas y zonas de cobertura geográfica asociadas al sistema. Esta clase es utilizada para establecer la ubicación física de clientes, servidores, antenas y routers dentro del sistema.

Métodos Principales:

- crearSede (Estático): Permite crear un rectángulo que representa la forma y los límites de una sede.
- crearZonaCobertura (Instancia): Genera una zona de cobertura circular que es utilizada por elementos como antenas.
- crearPuntoOrdenado (Instancia): Convierte coordenadas en un objeto Point2D para facilitar operaciones geométricas y cálculos.
- Los Getters y Setters: Métodos para acceder y modificar los atributos privados de la clase.

Relación con Otras Clases:

- Con Cliente: Los clientes se ubican dentro del plano y tienen coordenadas asignadas para determinar su ubicación-.
- Con Servidor: Los servidores también están ubicados en el plano, y su posición se determina en función de la sede y la cobertura de red.
- Con Antena: Las antenas tienen una zona de cobertura definida en el plano.
- Con Router: Los routers están ubicados dentro de la zona de cobertura definida en el plano, para proporcionar acceso a los clientes.
- Con ProveedorInternet: Los proveedores gestionan los límites y zonas de cobertura, las cuales se visualizan en el plano mediante las sedes y zonas de cobertura.

3.4. Plan.

La clase Plan describe los tipos de planes de servicio que los proveedores de internet ofrecen a los clientes. Los planes se dividen en tres categorías: BASIC, STANDARD y PREMIUM. Cada plan contiene información sobre la velocidad de subida, velocidad de bajada y el precio del servicio.

Métodos Principales:

- Getters: Que retornan listas con los detalles del plan, incluyendo velocidad de subida, velocidad de bajada y precio.

Relación con Otras Clases:

- Con ProveedorInternet: Los planes de servicio (BASIC, STANDARD, PREMIUM) son gestionados y ofrecidos a través de los proveedores.
- Con Cliente: Los clientes contratan planes, dependiendo de sus necesidades.
- Con Factura: La factura del cliente se genera según el plan contratado.
- Con Router: Los routers están asociados a los planes de servicio para proporcionar acceso a internet.
- Con Dispositivo: Los dispositivos de los clientes están vinculados al plan contratado para acceso a los servicios.

3.5. Factura.

La clase Factura registra los datos financieros y administrativos de los clientes. Esta clase es responsable de manejar información relacionada con los pagos atrasados, el precio total del servicio y la fecha de activación del cliente. Además, incluye métodos para verificar la validez de una factura, gestionar pagos y generar nuevas facturas.

Métodos Principales:

- verificarFactura (Instancia): Verifica si una factura tiene menos de tres pagos atrasados para determinar si es válida.
- generarFactura (Instancia): Genera una nueva factura basada en el plan de servicio contratado y el proveedor asociado al cliente.
- toString (Instancia): Devuelve una representación textual detallada de la factura, incluyendo el nombre del cliente, IP, pagos atrasados y precio total.
- Los Getters y Setters: Para acceder y modificar los atributos privados de la clase, como el usuario, la IP, los pagos atrasados, el precio y el mes de activación.

Relación con Otras Clases:

- Con Cliente: Cada factura está asociada a un cliente, quien es el responsable del pago del servicio que se le ha proporcionado según el plan contratado.
- Con ProveedorInternet: El proveedor emite las facturas a los clientes por los servicios que proporciona, considerando los planes y los recursos utilizados.
- Con Plan: El costo de la factura está determinado por el plan contratado por el cliente, ya sea BASIC, STANDARD o PREMIUM.
- Con Dispositivo: Los dispositivos asociados a un cliente pueden influir en el cálculo de la factura, especialmente si afectan el consumo de recursos o servicios adicionales.
- Con Router: El uso del servicio a través de los routers puede tener un impacto en el monto de la factura, dependiendo de las condiciones del contrato o el consumo.

3.6. Servidor.

La clase Servidor gestiona la infraestructura necesaria para las conexiones de red y la asignación de recursos en el sistema. Cada servidor está asociado a múltiples routers y tien

Métodos Principales:

- buscarServidores (Estático): Busca y retorna todos los servidores disponibles en una sede específica que no estén saturados.
- verificarAdmin (Instancia): Verifica si un proveedor tiene servidores registrados en el sistema y retorna estos servidores en todas las localidades.
- distanciasOptimas (Instancia): Calcula las distancias óptimas en las que un servidor debe ubicarse para maximizar la intensidad del flujo de red.
- adminServidor (Estático): Instancia un objeto servidor aleatorio que puede usarse para invocar métodos como verificarAdmin.
- Los Getters y Setters: Permiten acceder y modificar los atributos privados de la clase, como la sede, el porcentaje de eficiencia, el estado de saturación, el proveedor asociado, las coordenadas y la lista de routers.

Relación con Otras Clases:

- Con ProveedorInternet: Cada servidor está vinculado a un proveedor de internet y administra las conexiones de sus clientes.
- Con Router: Un servidor posee múltiples routers que se conectan directamente a los clientes.
- Con Plano: Define la ubicación geográfica del servidor para calcular distancias y zonas de cobertura.
- La clase Servidor asegura que los recursos de red se utilicen de manera eficiente, optimizando la calidad del servicio para los clientes mientras gestiona la capacidad máxima del sistema.

3.7. Router.

La clase Router maneja la conectividad de red entre los clientes y los servidores. y está asociado a un servidor y a una antena.

Métodos Principales:

- actualizarVelocidad (Instancia): Ajusta las velocidades de subida y bajada del cliente en función de los dispositivos conectados y si el cliente se encuentra dentro de la zona de cobertura de la antena asociada.
- actualizarPing (Instancia): Calcula y actualiza el valor del ping del router dependiendo de las velocidades actuales.
- test (Instancia): Realiza un diagnóstico general del router llamando a métodos como actualizarVelocidad y actualizarPing. También evalúa condiciones como saturación, cobertura y compatibilidad de generación.
- verificarDispositivos (Instancia): Identifica los dispositivos conectados al router en función de su IP.
- protocoloSeleccionServidor (Instancia): Evalúa las condiciones actuales del router y su servidor asociado para recomendar un servidor diferente si hay otro mejor o más cercano.
- adminRouter (Estático): Crea un router aleatorio que podrá ser utilizado por el administrador para pruebas y reportes
- Los Getters y Setters: Proporcionan acceso a los atributos privados de la clase, como la dirección IP, velocidades, estado en línea, coordenadas, servidor asociado, antena asociada y velocidad total.

Relación con Otras Clases:

- Con Servidor: Un router está directamente vinculado a un servidor, que le proporciona los recursos de red.
- Con Antena: Está asociado a una antena que define su zona de cobertura.
- Con Cliente: Permite la conectividad de los dispositivos del cliente al servidor y gestiona las velocidades asignadas según los planes contratados.

3.8. Dispositivo.

La clase Dispositivo representa los dispositivos conectados a la red. Cada dispositivo tiene información como su dirección IP, nombre, generación tecnológica, y está conectado a un router que gestiona su acceso a la red. Esta clase desempeña un papel importante en funcionalidades como visualizar dispositivos, realizar pruebas de red y optimizar planes de servicio.

Métodos Principales:

- desconectarDispositivo (Estático): Permite desconectar un dispositivo específico de la red asociada al cliente.
- toString (Instancia): Proporciona una representación textual del dispositivo, incluyendo su nombre, dirección IP y generación tecnológica.
- Los Getters y Setters: Proveen acceso a los atributos privados de la clase, como el router asociado, dirección IP, nombre, generación y la lista estática de dispositivos totales.

Relación con Otras Clases:

- Con Router: Cada dispositivo está conectado a un router que gestiona su conexión a la red.
- Con Cliente: Los dispositivos son propiedad de los clientes y están vinculados al router del cliente para acceder al servicio.

3.9. Antena.

La clase Antena gestiona la transmisión de señales dentro de una zona de cobertura específica. Cada antena está asociada a un proveedor de internet y se conecta con routers que distribuyen la señal.

Métodos Principales:

- antenasSede (Estático): Busca las antenas que se encuentran en una sede específica.
- antenasSede (Estático - Sobrecarga): Busca las antenas en una sede específica que pertenezcan a un proveedor determinado.
- rastrearGeneracionCompatible (Instancia): Identifica una antena dentro de una lista que sea compatible con la generación del router del cliente y cuya zona de cobertura incluya al cliente.
- verificarZonaCobertura (Instancia): Verifica si un router se encuentra dentro de la zona de cobertura de la antena.
- toString (Instancia): Proporciona una descripción textual de la antena, incluyendo su identificador, sede, ubicación y generación tecnológica.
- antenas (Estático): Resetea las referencias de los proveedores de las antenas al deserializar, para evitar problemas relacionados con referencias obsoletas.
- Los Getters y Setters: Proveen acceso a los atributos privados de la clase, como el identificador, coordenadas, sede, zona de cobertura y proveedor.

Relación con Otras Clases:

- Con ProveedorInternet: Cada antena está asociada a un proveedor, que define las características del servicio que ofrece.
- Con Router: Las antenas conectan los routers que distribuyen la señal a los clientes.
- Con Plano: Define la ubicación geográfica de la antena y su zona de cobertura.

3.10. Cobertura (Abstracta).

La clase Cobertura es abstracta, define las características generales relacionadas con la intensidad y la generación de flujo de red. Es una clase base que no puede ser instanciada directamente, pero proporciona una estructura común y métodos abstractos que son implementados por sus subclases, como Antena y Router.

Métodos Principales:

- rastrearGeneracionCompatible (Abstracto): es utilizado por la clase Antena en la funcionalidad de pruebas (test). Este método busca una antena compatible con la generación del router del cliente.
- intensidadFlujoOptima (Abstracto): Implementado por Router para la funcionalidad de optimización del plan de servicio. Calcula la intensidad de flujo óptima en función de los servidores y las características del cliente.
- intensidadFlujoClientes (Abstracto): Implementado por Router en la funcionalidad de reportes. Calcula la intensidad de flujo que los clientes deberían recibir en comparación con la intensidad real.
- toString (Sobrescrito): Devuelve una representación textual de la clase. Se puede personalizar en las subclases para mostrar información específica.
- Los Getters y Setters: Proporcionan acceso a los atributos privados de la clase:
- intensidadFlujo: Indica la intensidad de flujo calculada.

Relación con Otras Clases:

- Subclases: Antena y Router usan esta clase para implementar comportamientos específicos, en cada una.
- Con Cliente y Servidor: Colabora con estas clases para calcular la intensidad de flujo, rastrear antenas.
- Con Antena: Define los métodos necesarios para rastrear generaciones compatibles dentro de la cobertura.

3.11. Red (Interfaz).

La interfaz Red declara métodos y constantes para la optimización del flujo de red. Está diseñada para ser implementada por la clase Servidor, lo que asegura que cualquier servidor cumpla con las operaciones necesarias para gestionar y optimizar la red de manera eficiente. Incluye métodos para verificar la administración de proveedores y calcular distancias óptimas para mejorar la intensidad del flujo.

Métodos Principales:

- verificarAdmin (Abstracto): Verifica la existencia de servidores asociados a un proveedor específico dentro del sistema.
- Nombre del proveedor (String)
- distanciasOptimas (Abstracto): Calcula las distancias óptimas para ubicar servidores para mejorar la intensidad de flujo de red.
- FLUJO_RED_PRELIMINAR (Estático y Final): Define un valor base para la intensidad preliminar del flujo de red, utilizado como referencia en cálculos de optimización (Valor: 500).

Relación con Otras Clases:

- Implementada por Servidor: En la clase Servidor se utiliza esta interfaz para implementar métodos.
- Con Cliente y ProveedorInternet: Colabora con estas clases para analizar las intensidades de flujo y gestionar la red.

4. Descripción de la implementación de características de programación orientada a objetos en el proyecto.

4.1. Clase abstracta y método abstracto.

- Clase:

```
11 public abstract class Cobertura implements Serializable{
```

Ubicación: gestorAplicacion/enrutadorHFC/Cobertura, línea 11.

- Método: Implementamos tres métodos abstractos que deben ser implementados por las clases que heredan: Antena y Router. Cada método tiene un propósito específico y cambia dependiendo de la funcionalidad implementada.

```
26 //METODO UTILIZADO EN SU MAYORIA POR LA CLASE ANTENA PARA LA FUNCIONALIDAD DEL TEST
27 public abstract Antena rastrearGeneracionCompatible(ArrayList<Antena> antenasSede, Router r);
28
29 //METODO UTILIZADO EN SU MAYORIA POR LA CLASE ROUTER PARA LA FUNCIONALIDAD MEJORA TU PLAN
30 public abstract ArrayList<Object> intensidadFlujoOptima(ArrayList<Servidor> ss, Cliente c);
31
32 //METODO UTILIZADO EN SU MAYORIA POR LA CLASE ROUTER PARA LA FUNCIONALIDAD REPORTE
33 public abstract ArrayList<Integer> intensidadFlujoClientes(ArrayList<Cliente> ctes, Servidor servidor, boolean Reales);
```

Ubicación: gestorAplicacion/enrutadorHFC/Cobertura.java, línea 27-33.

Un ejemplo de la implementación sería en la clase Antena:

```
67 @Override
68 public Antena rastrearGeneracionCompatible(ArrayList<Antena> antenasSede, Router r){
69
70     List<Antena> antenasCercanas=antenasSede.stream().
71     filter(antenas -> antenas.verificarZonaCobertura(r,antenas)).collect(Collectors.toList());
72     ArrayList<Antena> antenaRecomendada=new ArrayList<Antena>();
73
74     Antena antenaEncontrada=antenasCercanas.stream().
75     filter(antenas->antenas.getGeneracion()==r.getGeneracion()).findFirst().orElse(other:null);
76
77     if(antenaRecomendada!=null){
78         antenaRecomendada.add(antenaEncontrada);
79         return antenaRecomendada.get(index:0);
80     }else{
81         return null;
82     }
83 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Antena.java, línea 68.

4.2 Interfaz (método default y método estático).

```

5  package gestorAplicacion.enrutadorHFC;
6
7  import gestorAplicacion.host.Cliente;
8  import gestorAplicacion.host.ProveedorInternet;
9  import java.util.ArrayList;
10
11  public interface Red{
12
13      //INTERFACE CREADA PARA QUE LA CLASE SERVIDOR LA IMPLEMENTE
14
15      //CONSTANTES
16      static final int FLUJO_RED_PRELIMINAR=500;
17
18      //MÉTODOS
19      //METODO POR DEFAULT-FUNCIONALIDAD REPORTE--VERIFICA ADMIN Y RETORNA SERVIDORES DE UN PROVEEDOR EN TODAS LAS LOCALIDADES
20      default ArrayList<Servidor> verificarAdmin(ArrayList<ProveedorInternet> proveedores, String nombre){
21          ArrayList<Servidor> servidoresProveedor = new ArrayList<>();
22          for(ProveedorInternet proveedor: proveedores){
23              if(proveedor.getNombre().equals(nombre)){
24                  for(Servidor servidor: Servidor.getServidoresTotales()){
25                      if(servidor.getProveedor().getNombre().equals(nombre)){
26                          servidoresProveedor.add(servidor);
27                      }
28                  }
29              }
30          }
31          return servidoresProveedor;
32      }
33      ArrayList<Integer> distanciasOptimas(ArrayList<Cliente> clientes,ArrayList<Integer> listaIntensidadesClientes);
34
35      //METODO ESTATICO-FUNCIONALIDAD REPORTE--CALCULA EL PROMEDIO DE LAS INTESIDADES NETAS---SE USA POR LIGADURA DINAMICA
36      static int calcularPromedioIntensidad(ArrayList<Integer> intensidades) {
37          return intensidades.stream().mapToInt(Integer::intValue).sum() / intensidades.size();
38      }
39  }

```

Ubicación: gestorAplicacion/enrutadorHFC/Red.java

Esta interfaz es usada por la clase servidor:

```

13  public class Servidor implements Red, Serializable{

```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java

Es fundamental para la funcionalidad de reporte (Funcionalidad 5), ya que permite listar y analizar los servidores que pertenecen a un proveedor, facilitando la generación del informe. En cuanto a la constante de la interfaz, está es para el flujo total que todo proveedor de Internet proporciona a cada uno de sus servidores.

- Método por default: verificarAdmin, recibe una lista de proveedores de Internet y un nombre, y retorna una lista de servidores asociados al proveedor cuyo nombre coincide con el especificado. Para ello, recorre la lista de proveedores y, si encuentra uno con el nombre indicado, busca en la lista global de servidores (Servidor.getServidoresTotales()) aquellos que pertenezcan a ese proveedor, agregándolos a la lista de resultados.

```

19  //METODO POR DEFAULT-FUNCIONALIDAD REPORTE--VERIFICA ADMIN Y RETORNA SERVIDORES DE UN PROVEEDOR EN TODAS LAS LOCALIDADES
20  default ArrayList<Servidor> verificarAdmin(ArrayList<ProveedorInternet> proveedores, String nombre){
21      ArrayList<Servidor> servidoresProveedor = new ArrayList<>();
22      for(ProveedorInternet proveedor: proveedores){
23          if(proveedor.getNombre().equals(nombre)){
24              for(Servidor servidor: Servidor.getServidoresTotales()){
25                  if(servidor.getProveedor().getNombre().equals(nombre)){
26                      servidoresProveedor.add(servidor);
27                  }
28              }
29          }
30      }
31      return servidoresProveedor;

```

Ubicación: gestorAplicacion/enrutadorHFC/Red.java, línea 20.

- Método estático: calcula el promedio de una lista de intensidades recibida como parámetro. Utiliza un flujo de datos (stream) para sumar los valores de la lista y divide el total entre la cantidad de elementos para obtener el promedio. Es un ejemplo de método estático porque está asociado a la clase en lugar de a una instancia específica, lo que permite que el método sea invocado directamente desde la clase sin necesidad de crear un objeto.

```
35 //METODO ESTATICO-FUNCIONALIDAD REPORTE--CALCULA EL PROMEDIO DE LAS INTESIDADES NETAS---SE USA POR LIGADURA DINAMICA
36 static int calcularPromedioIntensidad(ArrayList<Integer> intensidades) {
37     return intensidades.stream().mapToInt(Integer::intValue).sum() / intensidades.size();
38 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Red.java, línea 36.

4.3. Herencia.

Tenemos varios casos de herencia que nos ayudan a implementar las funcionalidades de manera eficiente:

- Herencia de la clase Cobertura: La clase Antena y Router heredan de la clase abstracta Cobertura. Esto nos permite aprovechar los atributos y métodos definidos en Cobertura.

```
14 public class Antena extends Cobertura{
```

Ubicación: gestorAplicacion/enrutadorHFC/Antena.java, línea 14.

```
11 public class Router extends Cobertura {
```

Ubicación: gestorAplicacion/enrutadorHFC/Router .java, línea 11.

Esta herencia asegura que tanto Antena como Router, definan cada uno sus propias implementaciones de los métodos, para las necesidades de cada clase.

- Herencia de la clase Router en Conexión: Lo que le permite acceder a todos los atributos y métodos de la clase Router, así conexión puede acceder directamente a los atributos y métodos de Router, cómo la IP, la velocidad de conexión, la cantidad de dispositivos conectados, etc., sin necesidad de crear una instancia de Router por separado.

```
9 public class Conexion extends Router{
```

Ubicación: gestorAplicacion/enrutadorHFC/Conexion .java, línea 9.

4.4. Ligadura dinámica.

- Caso 1: Método intensidadFlujoOptima, se crea un apuntador de la clase Router, pero se le asigna un objeto de la clase Conexión. Esto es posible porque Conexion hereda router. Luego, se llama al método getVelocidad, que está definido en Router. Sin embargo, como Conexión sobrescribe este método, se ejecuta la versión de Conexión, que realiza un cálculo más complejo en lugar de devolver simplemente el valor por defecto (250). Se llama:

```
304 Router conexion = new Conexion(IP, down, up, online, generacion, c, c.getModem().getServidorAsociado());
305 int velocidadTotal = conexion.getVelocidad();
```

Ubicación: gestorAplicacion/enrutadorHFC/Router.java, en la línea 305.

Se define:

```
447 public int getVelocidad() {
448     return velocidad;
449 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Router.java, línea 447.

Se sobrescribe:

```
25 public int getVelocidad() {
26     int velocidadActual = ((cliente.getModem().actualizarVelocidad(cliente).
27     get(index:0) + cliente.getModem().actualizarVelocidad(cliente).get(index:1)) / 2)*generacion*generacion;
28     return velocidadActual;
29 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Conexion.java, línea 25.

- Caso 2: método rastrearGeneraciónCompatible en la clase Main, se crea un apuntador de la clase Cobertura, que es abstracta, por lo que no se puede instanciar directamente. Sin embargo, se le asigna un objeto de la clase Antena, que hereda de Cobertura. Luego, se llama al método rastrearGeneraciónCompatible, que está definido como un método abstracto en Cobertura y es sobrescrito en Antena. Se llama:

```
781 //SE APLICA LIGADURA DINAMICA
782 Cobertura coberturaActual = clienteActual.getModem().getAntenaAsociada();
783 Antena antenaNueva = coberturaActual.rastrearGeneracionCompatible(antenasSede, clienteActual.getModem());
```

Ubicación: uiMain/Main.java, línea 783.

Se define:

```
26 //METODO UTILIZADO EN SU MAYORIA POR LA CLASE ANTENA PARA LA FUNCIONALIDAD DEL TEST
27 public abstract Antena rastrearGeneracionCompatible(ArrayList<Antena> antenasSede, Router r);
```

Ubicación: gestorAplicacion/enrutadorHFC/Cobertura.java, línea 27.

Se sobrescribe:

```
67 @Override
68 public Antena rastrearGeneracionCompatible(ArrayList<Antena> antenasSede, Router r){
69
70     List<Antena> antenasCercanas=antenasSede.stream().
71     filter(antenas -> antenas.verificarZonaCobertura(r,antenas)).collect(Collectors.toList());
72     ArrayList<Antena> antenaRecomendada=new ArrayList<Antena>();
73
74     Antena antenaEncontrada=antenasCercanas.stream().
75     filter(antenas->antenas.getGeneracion()==r.getGeneracion()).findFirst().orElse(other:null);
76
77     if(antenaRecomendada!=null){
78         antenaRecomendada.add(antenaEncontrada);
79         return antenaRecomendada.get(index:0);
80     }else{
81         return null;
82     }
83 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Antena.java, línea 68.

4.5. Ligadura estática.

La interfaz Red contiene el método calcularPromedioIntensidad, que se declara como static.

```
35 //METODO ESTATICO-FUNCIONALIDAD REPORTE--CALCULA EL PROMEDIO DE LAS INTESIDADES NETAS---SE USA POR LIGADURA DINAMICA
36 static int calcularPromedioIntensidad(ArrayList<Integer> intensidades) {
37     return intensidades.stream().mapToInt(Integer::intValue).sum() / intensidades.size();
38 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 36.

Esto implica que no importa si existen subclases de Red, el método siempre será el definido en la clase Red, ya que el llamado al método es resuelto en tiempo de compilación a través de la interfaz.

```
960 //PROMEDIO INTENSIDADES REALES
961 int PromedioBasic = Red.calcularPromedioIntensidad(intensidadBasic); //LIGADURA ESTATICA
962 int PromedioStandard = Red.calcularPromedioIntensidad(intensidadStandard); //LIGADURA ESTATICA
963 int PromedioPremium = Red.calcularPromedioIntensidad(intensidadPremium); //LIGADURA ESTATICA
```

Ubicación: uiMain/Main.java, línea 961-963.

4.6. Atributos de clase y métodos de clase.

- Atributos: el atributo servidoresTotales mantiene un registro de todos los servidores creados en el sistema.

```
23 private static ArrayList<Servidor> servidoresTotales = new ArrayList<>();
```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 23.

El atributo FLUJO_RED_PRELIMINAR es utilizado para definir el flujo de red, que los servidores puedan manejar, para hacer cálculos como la saturación y la capacidad de los servidores.

```
16 static final int FLUJO_RED_PRELIMINAR=500;
```

Ubicación: gestorAplicacion/enrutadorHFC/Red.java, línea 16.

- Métodos:

- El metodo buscarServidores, este método permite encontrar servidores en una localidad específica, utilizando los atributos estáticos como servidoresTotales.

```
55 public static ArrayList<Servidor> buscarServidores(String localidad) {
56     ArrayList<Servidor> serviSede = new ArrayList<Servidor>();
57     for (Servidor servidor : servidoresTotales) {
58         if ((servidor.getSede().equals(localidad)) && (!servidor.isSaturado())) {
59             serviSede.add(servidor);
60         }
61     }
62     return serviSede;
63 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 55.

- El método adminServidor() crea un objeto nuevo de la clase Servidor y lo devuelve. Se usa para generar un servidor de manera rápida, especialmente cuando se necesita llamar al método verificarAdmin() y es estático porque no necesita un objeto de Servidor para funcionar.

```
94 public static Servidor adminServidor(){
95     return new Servidor();
96 }
```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 94.

4.7. Uso de constante.

En el proyecto implementamos varias constantes en las clases Router, Servidor, Cliente, ProveedorInternet y Plan. No hablaremos de todas, pero algunas de las más relevantes. En la clase ProveedorInternet, se definen las constantes CLIENTES_MAX, CLIENTES_BASIC, CLIENTES_STANDARD y CLIENTES_PREMIUM. Estas constantes establecen los límites máximos de clientes que pueden estar en los diferentes planes que ofrece el proveedor, garantizando una gestión controlada de los recursos. CLIENTES_MAX establece el total de clientes que un proveedor puede atender, mientras que las otras tres constantes limitan la cantidad de clientes que pueden pertenecer a cada plan específico (BASIC, STANDARD y PREMIUM).

```
21 private final int CLIENTES_MAX;
22 private final int CLIENTES_BASIC;
23 private final int CLIENTES_STANDARD;
24 private final int CLIENTES_PREMIUM;
```

Ubicación: gestorAplicacion/host/ProveedorInternet.java, línea 21-24.

4.8. Encapsulamiento (Private, protected y public).

Las clases de la capa lógica de la aplicación se organizaron dentro de un paquete principal llamado gestorAplicación, tal como se indicó en el proyecto. Este paquete se subdividió en tres subpaquetes: enrutadorHFC, servicio y host, con base en las relaciones entre las clases.



El nivel de acceso a los atributos de las clases se configuró como 'privado', excepto en la clase Cobertura. Debido a que es una clase base, sus atributos se definieron como 'protected' para permitir que las clases derivadas accedan a ellos sin necesidad de usar los métodos 'get'.

```
14 protected int generacion;
15 protected int intensidadFlujo;
```

Ubicación: gestorAplicacion/enrutadorHFC/Cobertura.java, línea 14-15.

En cuanto a los métodos de las clases, se definieron con un nivel de acceso public para permitir su ejecución en otras clases que pueden estar en paquetes diferentes.

4.9. Sobrecarga.

- Métodos: En la clase proveedorInternet, existe una sobrecarga en el método, verificarCliente. Un método recibe (long id) y el otro recibe (String nombre, long id), una es para, verificar si ese cliente existe dentro del arreglo de clientes del proveedor. Si encuentra el cliente, también verifica que el cliente tenga un plan específico (como "PREMIUM" o "STANDARD") y, si es así, lo retorna. El otro método lo que hace es además de verificar si el cliente existe en el arreglo de clientes, también valida que el cliente tenga más de 2 dispositivos conectados a través de su módem. Si cumple con esta condición, retorna al cliente.

```

46 public Cliente verificarCliente(long id){
47
48     for (Cliente cliente : clientes) {
49         if (cliente.getID() == id) {
50             if (cliente.getNombrePlan().equals(anObject:"PREMIUM")){
51                 return cliente;
52             }else if(cliente.getNombrePlan().equals(anObject:"STANDARD")){
53                 return cliente;
54             }else{
55                 return null;
56             }
57         }
58     }
59     return null;
60 }

```

Ubicación: gestorAplicacion/host/ProveedorInternet.java, línea 46.

```

63 public Cliente verificarCliente(String nombre,long id){
64     for (Cliente cliente : clientes) {
65         if ((cliente.getID() == id) && (cliente.getNombre().equals(nombre))){
66             if((cliente.getModem().verificarDispositivos(Dispositivo.getDispositivosTotales(),cliente.getModem()).size())>2){
67                 return cliente;
68             }
69         }
70     }
71     return null;
72 }

```

Ubicación: gestorAplicacion/host/ProveedorInternet.java, línea 63.

En la clase Antena, existe una sobrecarga en el método antenasSede. Un método recibe (String sede), y el otro recibe (String sede, String proveedor). El primer método busca si hay antenas asociadas a una sede específica y, si las encuentra, las retorna. El segundo método, además de verificar la sede, valida que las antenas pertenezcan a un proveedor específico dentro de esa sede y, si es así, retorna la antena correspondiente.

```

41 public static ArrayList<Antena> antenasSede(String sede){
42     ArrayList<Antena> antenas = new ArrayList<>();
43     for(Antena antena: antenasTotales){
44         if(antena.getSede().equals(sede)){
45             antenas.add(antena);
46         }
47     }
48     return antenas;
49 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Antena.java, línea 41.

```

53 public static Antena antenasSede(String sede, String proveedor){
54     for(Antena antena: antenasTotales){
55         if(antena.getSede().equals(sede)){
56             if(antena.getProveedor().getNombre().equals(proveedor)){
57                 return antena;
58             }
59         }
60     }
61 }
62 return null;
63 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Antena.java, línea 5.

- Constructores: En la clase router, tenemos tres constructores y se evidencia la sobrecarga, el primer constructor asocia el router con un servidor, el segundo constructor lo asocia con una antena y también especifica sus coordenadas, y el tercer constructor permite crear un objeto Router con valores predeterminados.

```

28 public Router(int up, int down, boolean online, Servidor servidorAsociado) {
29
30     super(g:3);
31     Random random = new Random();
32     int octeto1 = random.nextInt(bound:256);
33     int octeto2 = random.nextInt(bound:256);
34     int octeto3 = random.nextInt(bound:256);
35     int octeto4 = random.nextInt(bound:256);
36     String ip = octeto1 + "." + octeto2 + "." + octeto3 + "." + octeto4;
37
38     IP = ip;
39     this.up = up;
40     this.down = down;
41     this.online = online;
42     sede=servidorAsociado.getSede();
43     this.servidorAsociado=servidorAsociado;
44     velocidad=250;
45     servidorAsociado.getRouters().add(this);
46 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Router.java, línea 28.

```

48 public Router(int up, int down, boolean online, Antena antenaAsociada,Plano coordenadas,int g) {
49
50     super(g);
51
52     Random random = new Random();
53     int octeto1 = random.nextInt(bound:256);
54     int octeto2 = random.nextInt(bound:256);
55     int octeto3 = random.nextInt(bound:256);
56     int octeto4 = random.nextInt(bound:256);
57     String ip = octeto1 + "." + octeto2 + "." + octeto3 + "." + octeto4;
58
59     IP = ip;
60     this.up = up;
61     this.down = down;
62     this.online = online;
63     this.coordenadas=coordenadas;
64     this.antenaAsociada=antenaAsociada;
65     velocidad=250;
66     sede=antenaAsociada.getSede();
67 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Router.java, línea 48.

```

69 public Router(){
70     super(g:3);
71     IP="";
72     velocidad=250;
73 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Router.java, línea 69.

Otra sobrecarga estaría en la clase Servidor, los tres constructores. El primer constructor permite configurar un servidor con toda la información disponible, incluyendo su ubicación y proveedor. El segundo constructor se enfoca en aspectos esenciales como saturación, capacidad y eficiencia. Finalmente, el constructor por defecto es útil para crear objetos de manera rápida, con valores iniciales que se podrán modificar posteriormente.

```

29 public Servidor(String sede, boolean saturado, ProveedorInternet proveedor, Plano coordenadas, int indice, double pe) {
30     this(saturado,indice,pe);
31     this.sede = sede;
32     this.proveedor = proveedor;
33     this.coordenadas = coordenadas;
34
35 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 29.

```

37 public Servidor(boolean saturado, int INDICE_SATURACION, double pe){
38     this.saturado=saturado;
39     this.INDICE_SATURACION=INDICE_SATURACION;
40     PORCENTAJE_EFICIENCIA = pe;
41     this.FLUJO_RED_NETO=(int)(PORCENTAJE_EFICIENCIA*FLUJO_RED_PRELIMINAR);
42     servidoresTotales.add(this);
43 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 37.

```

45 public Servidor(){
46     INDICE_SATURACION=0;
47     PORCENTAJE_EFICIENCIA = 0;
48     FLUJO_RED_NETO=0;
49 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 45.

4.10. Referencias del this y this().

- Uso del **this** para desambiguar: en la clase router se usa para distinguir los atributos de la clase (up, down, online, servidorAsociado) de los parámetros del constructor con el mismo nombre.

```

28 public Router(int up, int down, boolean online, Servidor servidorAsociado) {
29
30     super(g:3);
31     Random random = new Random();
32     int octeto1 = random.nextInt(bound:256);
33     int octeto2 = random.nextInt(bound:256);
34     int octeto3 = random.nextInt(bound:256);
35     int octeto4 = random.nextInt(bound:256);
36     String ip = octeto1 + "." + octeto2 + "." + octeto3 + "." + octeto4;
37
38     IP = ip;
39     this.up = up;
40     this.down = down;
41     this.online = online;
42     sede=servidorAsociado.getSede();
43     this.servidorAsociado=servidorAsociado;
44     velocidad=250;
45     servidorAsociado.getRouters().add(this);
46 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Router.java, línea 28.

Similar al caso anterior, this se usa para desambiguar entre los parámetros y los atributos de la clase (cliente).

```

30 public Cliente(String nombre, long id, Router modem, ProveedorInternet proveedor, ArrayList<Integer> plan, String nombrePlan, Factura factura) {
31     this.ID= id;
32     this.nombre=nombre;
33     this.modem = modem;
34     this.proveedor = proveedor;
35     this.plan = plan;
36     this.nombrePlan = nombrePlan;
37     this.factura = factura;
38     proveedor.getClientes().add(this);
39 }

```

Ubicación: gestorAplicacion/host/Cliente.java, línea 30.

- Uso del **this()**: lo utilizamos para invocar otro constructor sobrecargado de la misma clase, this(saturado, indice, pe).

Se asignan los resultados.

```

29 public Servidor(String sede, boolean saturado, ProveedorInternet proveedor, Plano coordenadas, int indice, double pe) {
30     this(saturado, indice, pe);
31     this.sede = sede;
32     this.proveedor = proveedor;
33     this.coordenadas = coordenadas;
34
35 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java, línea 29.

```

37 public Servidor(boolean saturado, int INDICE_SATURACION, double pe){
38     this.saturado=saturado;
39     this.INDICE_SATURACION=INDICE_SATURACION;
40     PORCENTAJE_EFICIENCIA = pe;
41     this.FLUJO_RED_NETO=(int)(PORCENTAJE_EFICIENCIA*FLUJO_RED_PRELIMINAR);
42     servidoresTotales.add(this);
43 }

```

Ubicación: gestorAplicacion/enrutadorHFC/Servidor.java , línea 37.

Otro Ejemplo se implementó fue en el constructor de ProveedorInternet, funciona igual que el ejemplo anterior solo que con 4 datos.

```

29 public ProveedorInternet(String nombre, int clientes_max, ArrayList<Integer> b, ArrayList<Integer> s, ArrayList<Integer> p, int cmb, int cms, int cmp) {
30     this(clientes_max, cmb, cms, cmp);
31     this.nombre = nombre;
32     this.planes = new Plan(b, s, p);
33 }

```

Ubicación: gestorAplicacion/host/ProveedorInternet.java, línea 29.

```

35 public ProveedorInternet(int clientes_max, int clientes_basic, int clientes_standard, int clientes_premium){
36     this.CLIENTES_MAX=clientes_max;
37     this.CLIENTES_BASIC=clientes_basic;
38     this.CLIENTES_STANDARD=clientes_standard;
39     this.CLIENTES_PREMIUM=clientes_premium;
40     proveedoresTotales.add(this);
41 }

```

Ubicación: gestorAplicacion/host/ProveedorInternet.java, línea 35.

4.11. Implementación de un caso de enumeración.

En nuestro proyecto, decidimos utilizar una enumeración para representar los meses del año. La clase Mes se definió como una enumeración con los doce meses, lo que nos permitió garantizar que los valores del atributo mesActivacion en la clase Factura sean siempre válidos y consistentes.

```

9 public enum Mes{
10     ENERO, FEBRERO, MARZO, ABRIL, MAYO, JUNIO, JULIO, AGOSTO, SEPTIEMBRE, OCTUBRE, NOVIEMBRE, DICIEMBRE;
11 }

```

Ubicación: gestorAplicacion/servicio/Mes.java, línea 9.

de Vcontrol1 a verdadero y se procederá con la implementación del switch. Al recibir la opción ingresada por el usuario, ejecutará la funcionalidad requerida.

```
50     if (opc == 1 || opc == 2 || opc == 3 || opc == 4 || opc == 5 || opc == 6) { //SE VERIFICA QUE LA OPCION ELEGIDA SEA
51
52         Vcontrol1 = true;
53
54         switch (opc) {
55
```

5.1. Funcionalidad 1: Adquisición de un plan.

La funcionalidad de adquisición de un plan tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia. En ella interactúan objetos de clases como 'Cliente', 'Factura' y 'ProveedorInternet' que mediante diferentes métodos se relacionan con objetos tipo 'Router', 'Servidor' y 'Antena'. A continuación, se detalla el funcionamiento del código paso a paso:

Al iniciar esta funcionalidad, se le piden por consola algunos datos al usuario: nombre (el string ingresado se tomará en minúsculas), cédula (tipo long), nombre de la sede (la letra ingresada como string se tomará en mayúscula), que se capturan con la ayuda del objeto scanner y su apuntador sc.

Se inicializa un while con Vcontrol15, que en esencia, funciona igual que el de Vcontrol1; es la base del resto de la funcionalidad y no permite la finalización del sistema si el usuario ingresa la letra de una sede inexistente (las opciones correctas son A, B, C, D).

Una vez se valide la sede, se listarán e imprimirán por consola los proveedores disponibles según la localidad del usuario para que se elija alguno y continuar el proceso:

```
Los proveedores disponibles en tu sede son:
1. tigo
2. claro
3. movistar
Ingresa la opción que deseas elegir: █
```

(ejemplo específico para los proveedores de la sede A).

El [primer método](#) usado en esta funcionalidad permite conocer los proveedores específicos de una sede: el método estático proveedorSede de la clase ProveedorInternet.

```
78     ArrayList<ProveedorInternet> proveedorDispo = ProveedorInternet.proveedorSede(sede);
```

Se comprueba que el número de opción ingresado sea válido (con el while de valPlan), en caso de no serlo, se solicita nuevamente. El [segundo método](#) usado en esta funcionalidad (gracias a crearClienteNuevo del proveedorActual) crea un objeto Cliente con el nombre e ID del usuario y el proveedor asignado.

```
109     Cliente clienteActual = proveedorActual.crearClienteNuevo(nombre, ID);
```

Se verifica si el proveedor elegido tiene cupos disponibles para nuevos clientes. Para ello, se accede al atributo clientes del objeto proveedorActual y se compara su tamaño actual con el valor definido en el atributo CLIENTES_MAX. Este proceso asegura que el número de clientes registrados no supere la capacidad máxima del proveedor.

En caso de que no haya cupos disponibles, se le notifica al usuario mediante un mensaje en

consola y se redirige al menú inicial. En caso de que sí, se mostrarán los planes disponibles del proveedor con sus características.

Esto se logra invocando el **tercer método** de esta funcionalidad: `planesDisponibles` a través del apuntador `proveedorActual`, pasando como argumento el cliente recién creado.

```
114 ArrayList<String> planesDisponibles = proveedorActual.planesDisponibles(clienteActual);
```

Este método retorna una lista de tipo `String` con los nombres de los planes disponibles para el cliente, validando que el número de clientes en cada plan no exceda el límite permitido para dicho plan.

Por cada plan disponible, se imprimen por consola las características clave, incluyendo la velocidad de subida y de bajada (en megas) y el precio, proporcionando al usuario una visión detallada:

```
Los planes disponibles del proveedor son:
BASIC
El plan basic ofrece:
Megas de subida: 10
Megas de bajada: 30
El precio del plan: 30000
STANDARD
El plan standard ofrece:
Megas de subida: 30
Megas de bajada: 50
El precio del plan: 55000
PREMIUM
El plan premium ofrece:
Megas de subida: 80
Megas de bajada: 100
El precio del plan: 87000
```

(Ejemplo específico para la sede A, proveedor CLARO).

Posteriormente, se solicita al usuario que ingrese el nombre del plan que desea adquirir (este proceso se realiza dentro de un bucle `while` que garantiza la validez del dato ingresado antes de continuar). El nombre del plan se toma en mayúsculas y debe coincidir con uno de los nombres válidos predefinidos (BASIC, STANDARD o PREMIUM).

Del mismo modo, debe estar disponible para el cliente según la lista generada por el método `planesDisponibles` del objeto `proveedorActual`. En caso de que el nombre ingresado sea incorrecto o el plan no esté disponible, el programa muestra un mensaje en consola solicitando una nueva entrada. Cuando logre validarse, el nombre del plan seleccionado se almacenará en la variable `planEscogido`.

Con la sede seleccionada anteriormente por el cliente, se definieron los límites de las coordenadas en los ejes X e Y, utilizando una estructura condicional `if-else`. Estos límites corresponden a un rango específico asociado a cada sede (A, B, C o D).

Por ejemplo:

En la sede A, las coordenadas X están limitadas entre 0 y 40, y las Y entre 0 y 40.

En la sede B, las coordenadas X están limitadas entre 0 y 40, y las Y entre 60 y 100.

Una vez determinados los límites, se solicita al usuario que ingrese sus coordenadas de ubicación. En consola se vería así:

```

Digita las coordenadas de tu ubicación, estas tienen un límite de acuerdo con tu sede.
Coordenadas en el eje X:
Límite Inferior: 0
Límite Superior: 40
Coordenadas en el eje Y:
Límite Inferior: 0
Límite Superior: 40
Ingresa la coordenada X:
Ingresa la coordenada Y:

```

(Ejemplo específico para la sede A).

Este ingreso se valida, una vez más, gracias a un bucle while que asegura que las coordenadas ingresadas estén dentro de los límites establecidos para la sede correspondiente. Si alguna coordenada está fuera de rango, se muestra un mensaje en consola indicando el error, y se solicita al usuario que las ingrese nuevamente.

Cuando las coordenadas son válidas, se almacenan en las variables coordenadaX y coordenadaY. Estos datos, junto a planEscogido, se utilizarán para configurar el plan seleccionado por el cliente utilizando el método configurarPlan (que pertenece al objeto clienteActual) y es el [cuarto método](#) de esta funcionalidad.

```

197 Factura facturaCliente = clienteActual.configurarPlan(planEscogido, clienteActual, sede, coordenadaX, coordenadaY);

```

Se asociarán las características seleccionadas al cliente, crear y configurar un objeto Router. Al enlazarlos con los objetos Servidor y Antena correspondientes según la sede y el proveedor, se retornará un objeto de tipo Factura, asociado al cliente.

Una vez generado el objeto Factura, se verifica que este sea válido utilizando una instancia de comprobación. Si la factura es válida, se notifica al cliente a través de un mensaje en consola indicando que el plan ha sido configurado correctamente. Posteriormente, se llama al método generarFactura (crea el detalle completo de la factura con la información del cliente y proveedor), es el [quinto método](#) de esta funcionalidad.

```

201 facturaCliente = facturaCliente.generarFactura(clienteActual, proveedorActual);

```

Este detalle es impreso por consola para que el cliente pueda visualizarlo:

```

Tu plan ha sido configurado correctamente. A continuación puedes visualizar tu factura.
Factura
Usuario: justin
Cédula: 1314
IP: 34.0.91.183
Mes de Activación: ENERO
Número de pagos atrasados: 0
Precio total a pagar: 87000

```

(Ejemplo específico de la Factura).

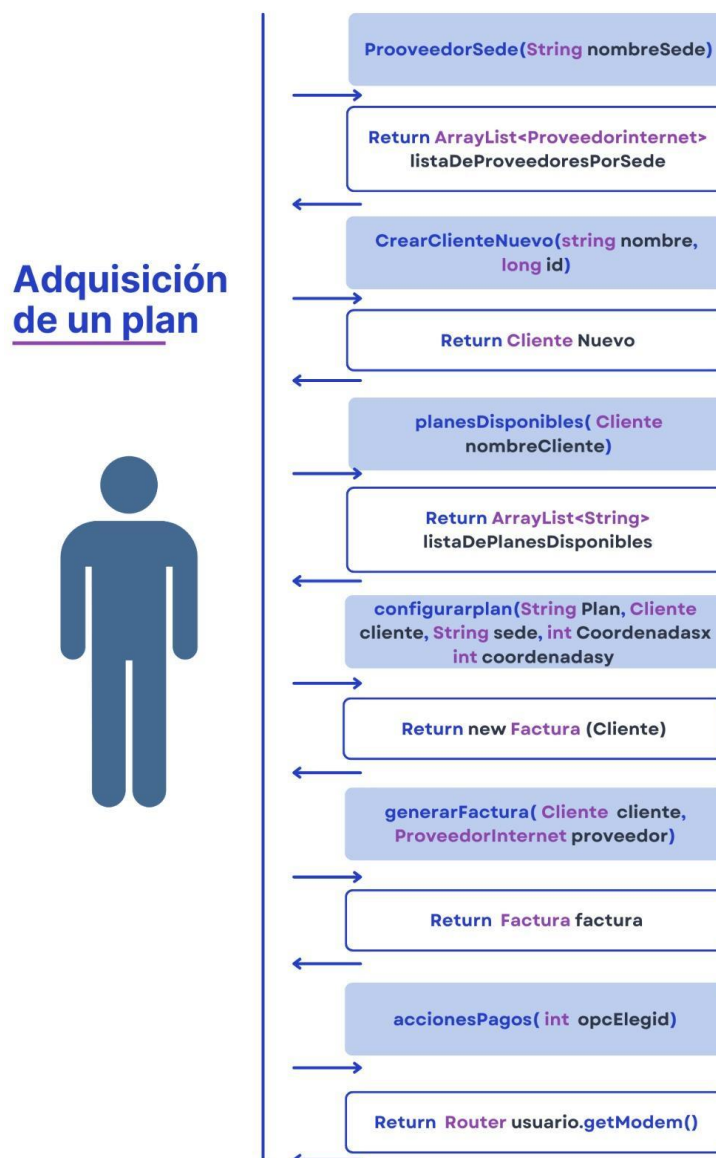
A continuación, se solicita al cliente realizar el pago correspondiente a la factura generada. Esto se implementa mediante un bucle while que valida si el pago ingresado es suficiente para cubrir el costo total del plan. Durante este proceso, si el cliente no ingresa el monto correcto, se muestra un mensaje en consola solicitando que complete la totalidad del pago. Una vez que el pago es validado correctamente, se informa al cliente que la adquisición del plan ha finalizado exitosamente, y el ciclo del proceso termina.

```
Realiza el pago de tu membresía a continuación (Ingresa el valor a pagar): 8700
Por favor ingresa la totalidad del precio requerido: 87000
Tu adquisición del plan ha finalizado con éxito.
```

(Ejemplo específico del pago de la totalidad de la factura).

En el caso de que el método configurarPlan no genere un objeto válido de tipo Factura, se informa al cliente que el plan no pudo ser configurado y se le indica que intente nuevamente en otro momento. Esto también ocurre si el proveedor no tiene cupos disponibles o si el usuario ingresa datos incorrectos, como una sede no válida. En cualquiera de estos casos, el proceso se interrumpe y el cliente es devuelto al menú inicial.

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad.



5.2. Funcionalidad 2: Visualizar los dispositivos conectados.

La funcionalidad de visualización de dispositivos conectados permite al cliente consultar una lista de dispositivos vinculados a su router, mostrando sus características principales. Está

disponible únicamente para clientes con planes 'Standard' o 'Premium' y con un máximo de dos pagos atrasados. En caso de exceder este límite, se ofrecen opciones para saldar o abonar la deuda antes de acceder a la funcionalidad. Este proceso involucra objetos de las clases 'Dispositivo', 'Router', 'Factura' y 'Cliente'. A continuación, se detalla el funcionamiento del código paso a paso:

Primero, se solicitan al usuario su identificación (ID) y el nombre de su proveedor. Así:

```
Ingresa tu Id:
1314
Ingresa el nombre de tu proveedor:
tigo
```

(Ejemplo específico de una entrada de datos).

Con esta información, el sistema busca en los datos de los proveedores registrados para verificar si el cliente existe. Este proceso se realiza recorriendo la lista de proveedores disponibles y comparando el nombre con el de cada proveedor. Si se encuentra un proveedor coincidente, se obtiene el objeto y se asigna al apuntador proveedorActual para continuar con las verificaciones.

Luego, con el proveedor identificado, se llama al método verificarCliente, el [primer método](#) de esta funcionalidad, para validar si el cliente está registrado.

```
260 Cliente clienteActual = proveedorActual.verificarCliente(ID);
```

Si el cliente no se encuentra en los datos del proveedor o tiene un plan del tipo "Basic", el sistema muestra un mensaje indicando que la funcionalidad no está disponible y regresa al menú principal. Si el cliente es válido, el proceso continúa con la verificación de su estado de pagos.

Si el cliente tiene más de dos pagos atrasados, el método verificarFactura, el [segundo método](#) de esta funcionalidad, lo indica mediante un retorno de false. En este caso, el sistema ofrece opciones para saldar o abonar a la deuda mediante el método accionesPagos.

```
289 Router routerCliente = clienteActual.getFactura().accionesPagos(opc);
```

Este método procesa el pago según la opción seleccionada (abonar o pagar la totalidad), actualiza el estado de la factura y verifica si el cliente ahora cumple con la condición de tener dos pagos atrasados o menos.

```
Tienes 7 pagos atrasados, para acceder a esta función debes tener 2 o menos pagos atrasados.
Tu factura actualmente está por un valor de $385000, recuerda que el valor mensual de tu plan es de $55000
Tenemos las siguientes opciones de pago:
1. Abonar a la deuda.
2. Pagar la totalidad de la deuda.
3. Regresar al menú inicial.
```

(Ejemplo específico de un usuario con +2 deudas acumuladas).

Si esta condición no se cumple después del pago, el proceso termina y el sistema regresa al menú principal.

Si el cliente cumple con las condiciones necesarias, se accede a la funcionalidad principal. Con el objeto Router del cliente, se llama al [tercer método](#) de esta funcionalidad, verificarDispositivos.

```
300 ArrayList<Dispositivo> lista = routerCliente.verificarDispositivos(Dispositivo.getDispositivosTotales(), routerCliente);
```

Este método recibe como parámetros la lista total de dispositivos y el router del cliente. Filtra los dispositivos que están vinculados al router mediante la comparación de las direcciones IP de ambos elementos. El resultado es una lista de dispositivos conectados al router del cliente.

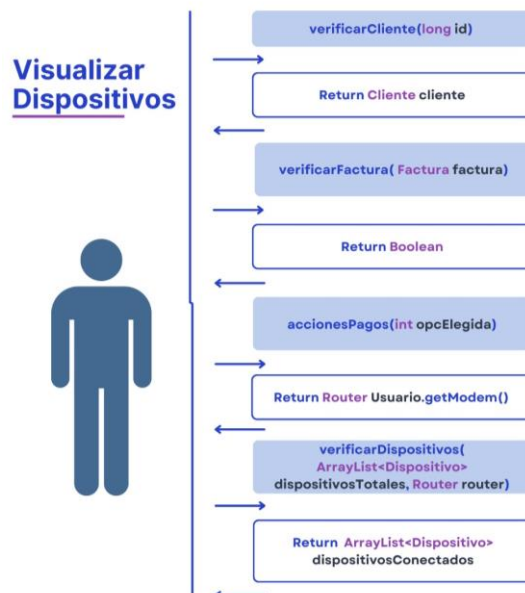
Finalmente, el sistema ofrece opciones para visualizar los dispositivos conectados por generación (3G, 4G o 5G) o mostrar todos los dispositivos conectados. Así en la consola:

```
¡Bienvenido!
Aquí podrás visualizar los dispositivos conectados a tu router.
Digita por favor la opción que deseas:
1. Visualizar dispositivos de 3G.
2. Visualizar dispositivos de 4G.
3. Visualizar dispositivos de 5G.
4. Visualizar todos los dispositivos conectados.
5. Regresar al menú inicial.
```

Según la opción seleccionada, se recorre la lista filtrada. Si no se encuentran dispositivos de la generación solicitada, el sistema informa al cliente que no hay dispositivos con esa característica. Si el cliente selecciona una opción válida, se muestran los dispositivos correspondientes. En caso de que el cliente elija una opción inválida, el sistema le solicita nuevamente que ingrese una opción válida.

Después de mostrar los dispositivos, si el cliente desea regresar al menú principal, el sistema le permite hacerlo. Si el cliente no elige la opción de regresar, finaliza la funcionalidad.

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad.



5.3. Funcionalidad 3: Mejora tu plan.

La funcionalidad Mejora tu plan recomienda al cliente un plan más adecuado basado en las características de su plan actual, considerando cálculos que involucran objetos de las clases 'Servidor', 'Cliente', 'Conexión' y 'Router', como distancias, intensidades de flujo y precios.

Está disponible solo para usuarios registrados con un plan adquirido y permite cambios únicamente dentro del mismo tipo de plan (por ejemplo, de Basic a otro Basic). Es ideal para clientes con al menos 3 dispositivos conectados que busquen acceder a mejores beneficios.

```
Digita por favor el número de la opción que deseas: 3
Ingresa tu Id:
1234
Ingresa tu nombre:
andrea
Ingresa el nombre de tu proveedor:
tigo
```

(Ejemplo específico de una entrada de datos).

A continuación, se detalla el funcionamiento del código paso a paso:

Primero, se solicitan al usuario algunos datos básicos: ID, nombre, nombre del proveedor. Estos dos primeros se usan como parámetros para verificar su registro mediante el [primer método](#) de esta funcionalidad, `verificarCliente`, de la clase `ProveedorInternet`, invocado desde un objeto `ProveedorInternet`. Si los datos ingresados son correctos, el método retorna un objeto `Cliente`; de lo contrario, se solicita nuevamente la información.

```
531     Cliente clienteActual = proveedorActual.verificarCliente(nombre, ID);
```

Se utiliza el método estático `buscarServidores` de la clase `Servidor`, que recibe como parámetro la localidad del Router asociado al usuario (un string). Este [segundo método](#) de la funcionalidad 3, recorre el arreglo de servidores totales (atributo estático de la clase `Servidor`) y almacena en un nuevo arreglo aquellos que coincidan con la localidad y no estén en estado saturado / llenos (atributo). El método retorna este arreglo de servidores disponibles en la sede, algo útil para los pasos siguientes.

```
533     ArrayList<Servidor> listaServLoc = Servidor.buscarServidores(localidad);
```

Luego, se invoca al [tercer método](#) de la funcionalidad: `protocoloSeleccionServidor` de la clase `Router`, utilizando el módem (objeto) asociado al usuario y pasando como parámetros el arreglo de servidores disponibles (resultado obtenido con el método anterior) y el objeto `Cliente`. Este método obtiene las coordenadas del router (atributo de `Router`), los dispositivos conectados mediante el método `verificarDispositivos` (resultado obtenido con la funcionalidad 2).

```
534     ArrayList<Object> mejoraIdeal = clienteActual.getModem().protocoloSeleccionServidor(listaServLoc, clienteActual);
```

Se calcula la distancia entre el router y su servidor asociado, determina la velocidad total de conexión con un objeto `Conexión` y, finalmente, calcula la intensidad de flujo mediante la siguiente fórmula que integra estos elementos y se encuentra en el método de instancia `protocoloSeleccionServidor` anterior.

```
199     intensidadFlujo = (int)((getServidorAsociado().getFLUJO_RED_NETO()+
200     (velocidadTotal / dispositivosConectados.size()) - distancia)*getServidorAsociado().getPORCENTAJE_EFICIENCIA());
```

El método `protocoloSeleccionServidor` (método principal) llama a `intensidadFlujoOptima` (otro método) para identificar el servidor con la mejor intensidad de flujo y cercanía dentro de la localidad del usuario (se realiza el mismo procedimiento utilizado anteriormente pero ahora para todos). Esto pasa en una línea del método `protocoloSeleccionServidor` en `Router`:


```
205    ArrayList<Object> intensidadMayor = intensidadFlujoOptima(servidores, c);
```

Este método intensidadFlujoOptima recibe el arreglo de servidores de la sede (el que recibe el método principal) y a Cliente como parámetros, y, retorna un arreglo de tipo Object (que contiene el objeto Servidor) y el valor de la intensidad de flujo mayor.

Una vez se obtienen estos valores se comparan con lo que actualmente posee el usuario, teniendo en cuenta los criterios de verificación en los posibles casos (definidos en Router):

1. Misma intensidad de flujo y mismo servidor: No se le recomienda un cambio al usuario, ya que ya tiene el mejor plan, uno ideal dentro de sus posibilidades.

```
213    if ( intensidadServidorAproximado == intensidadFlujo) {
214        if (servidorAproximado == servidorAsociado) {
215            return null;
216        }
    }
```

(Código completo comparaciones ítem 1 en protocoloSeleccionServidor de Router).

2. Misma intensidad de flujo y diferente servidor: Se comprueba que el cambio sea dentro del mismo tipo de plan, se comparan precios y se sugiere una mejora más económica, en caso de ser posible. Se retorna un ArrayList de tipo Object (formado con la intensidad de flujo actual y la mejor opción encontrada de plan, su servidor, su proveedor y las características que lo describen).

```
213    } else { //CUANDO LAS INTENSIDADES COINCIDEN PERO EL SERVIDOR ES DISTINTO
214        if (planCliente.equals(anObject:"BASIC")){
215            if(c.getPlan().get(index:2)>servidorAproximado.getProveedor().getPlanes().getBASIC().get(index:2)){
216
217                características.add(intensidadFlujo);
218                características.add(servidorAproximado);
219                características.add(servidorAproximado.getProveedor());
220                características.add(intensidadServidorAproximado);
221                características.add(servidorAproximado.getProveedor().getPlanes().getBASIC());
222
223                return características;
224            }
225        }
```

(Primera parte del código de comparaciones ítem 2 en protocoloSeleccionServidor).

3. Mayor intensidad de flujo en el servidor recomendado: El proceso es igual que el del ítem 2, pero no se comparan los precios de los planes, se recomienda el cambio directamente, priorizando la mejora en intensidad de flujo. Se retorna un ArrayList de tipo Object igual al del ítem anterior.

```
255    } else if (intensidadFlujo < intensidadServidorAproximado) { //CUANDO LA INTENSIDAD ACTUAL ES MENOR A LA CALCULADA
256        if (planCliente.equals(anObject:"BASIC")) {
257
258            características.add(intensidadFlujo);
259            características.add(servidorAproximado);
260            características.add(servidorAproximado.getProveedor());
261            características.add(intensidadServidorAproximado);
262            características.add(servidorAproximado.getProveedor().getPlanes().getBASIC());
263
264            return características;
265        }
```

(Primera parte del código de comparaciones ítem 3 en protocoloSeleccionServidor).

Una vez obtenidos los resultados, se presentan al usuario junto con las características de su plan actual, permitiendo una comparación entre ambos. Luego, se le da la opción de realizar el cambio, pero antes de proceder, se verifica si tiene pagos atrasados. Si los tiene, se le redirige a realizar el pago total de la deuda, lo que es necesario para completar exitosamente el cambio, se utiliza el método accionesPagos(opcion2), el [cuarto método](#) de esta

funcionalidad.

```
572     if (clienteActual.getFactura().accionesPagos(opcElegida:2) != null) {
```

(Primera parte del código del método).

Tras la cancelación de la deuda, se actualizan los objetos correspondientes y se finaliza el proceso de la funcionalidad.

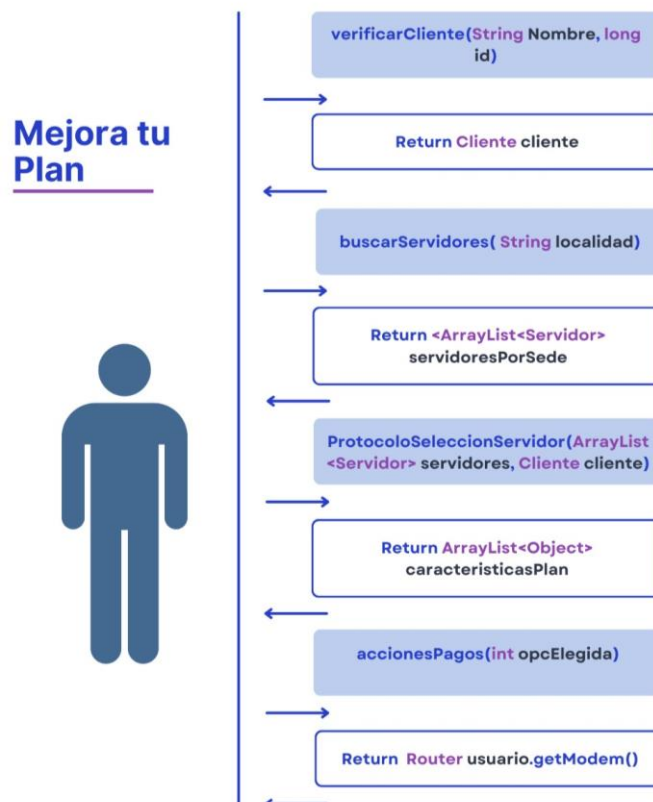
```
¿Deseas cambiar tu plan por la recomendación generada?
1. Sí
2. No
1

Tienes 7 pagos atrasados, para continuar debes saldar la deuda.
Tu factura actualmente está por un valor de $385000, elige una opción:
1. Pagar la totalidad de la deuda.
2. Salir.
1
Digite por favor la cantidad total a pagar:
385000
Tu plan ha sido actualizado con éxito.
```

(Ejemplo específico de una mejora de plan con el tipo de comparación 3 y pago de deudas).

En caso de que no tenga pagos atrasados, directamente se pasa a realizar los cambios respectivos y finalizar la funcionalidad.

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad.



5.4. Funcionalidad 4: Test.

El propósito de esta funcionalidad es permitir a los usuarios realizar pruebas de velocidad en su conexión de red. Esto incluye evaluar las velocidades de carga y descarga, así como medir el tiempo de respuesta o ping (milisegundos). Adicionalmente, se ofrece información sobre el estado de la conexión de la antena, como su cobertura y generación de señal, para que el usuario tenga una visión clara del rendimiento de su red y pueda identificar problemas o áreas de mejora. En la implementación participan objetos de tipo Antena, Router, Cliente y ProveedorInternet, los cuales interactúan para llevar a cabo las acciones necesarias. A continuación, se detalla el funcionamiento del código paso a paso:

El programa comienza solicitando al usuario que ingrese su información básica: su ID, nombre y el nombre de su proveedor. Estos datos son esenciales para verificar si el cliente está registrado en la base de datos de algún proveedor. La información ingresada se utiliza posteriormente para instanciar los objetos necesarios y realizar validaciones.

```
Ingresar tu Id:
0
Ingresar tu nombre:
COB
Ingresar el nombre de tu proveedor:
TIGO
```

(Ejemplo específico de una entrada de datos).

Una vez obtenidos los datos del usuario, se intenta localizar al proveedor correspondiente utilizando un bucle que recorre la lista de proveedores totales disponibles. Si el proveedor ingresado coincide con alguno registrado, el programa procede a verificar si el cliente existe en la base de datos de dicho proveedor. Esto se realiza con el [primer método](#) de esta funcionalidad: `verificarCliente`, que recibe como parámetros el nombre y el ID del cliente.

```
702 if (proveedorActual.verificarCliente(nombre, ID) != null) {
```

Si el cliente es encontrado, se crea una instancia de tipo Cliente, asignada a la variable `clienteActual`, pero, si el cliente no se encuentra en la base de datos, el método retornará un valor null y el usuario deberá proporcionar nuevamente datos válidos para proceder con la operación.

Luego, el programa realiza un test de conexión invocando al método `test` de la clase Router, que está asociado al cliente. Este, el [segundo método](#) de esta funcionalidad, recibe como argumento un objeto de tipo Cliente que corresponde al `clienteActual` y evalúa diversos parámetros como las velocidades de subida y bajada, la cantidad de dispositivos conectados, y otros factores que determinan el estado de la red. El resultado se almacena en una variable de tipo String llamada `resultado`.

```
706 String resultado = clienteActual.getModem().test(clienteActual);
```

Paralelamente, se genera una lista de antenas disponibles en la sede del cliente utilizando el método estático `antenasSede` de la clase Antena. Este [tercer método](#) de la funcionalidad, recibe a un string (que se obtiene de Cliente e indica la sede donde se ubica el `clienteActual`) como parámetro y devuelve una lista de antenas disponibles en dicha ubicación.

```
707 ArrayList<Antena> antenasSede = Antena.antenasSede(clienteActual.getModem().getSede());
```

Esta información es fundamental para obtener información detallada sobre el estado de la red del cliente, lista de antenas disponibles en la sede y analizar posibles soluciones en caso de problemas de red.

Con el resultado del test, el programa evalúa diferentes escenarios utilizando una serie de condicionales. Cada uno de estos condicionales aborda un posible estado de la red y propone soluciones específicas en relación al estado actual de la red del usuario.

– Primer caso: Si el resultado indica que la red está saturada (mediante el método `contains()` de la clase `String`), se ejecuta el bloque correspondiente, lo que señala que la cantidad de dispositivos asociados a la red del clienteActual supera la capacidad del plan. Se sugiere al usuario eliminar dispositivos y se muestra una lista en columna con los dispositivos asociados, permitiendo seleccionar cuáles desconectar para reducir la carga en el sistema y mejorar el rendimiento de la red. El programa solicita al usuario que seleccione el dispositivo a eliminar y, una vez confirmado, actualiza la configuración del cliente.

```

710 if (resultado.contains(s:"saturada")) {
711     System.out.println(resultado);
712     System.out.println(x:"Te recomendamos remover dispositivos");
713     System.out.println(x:"Dispositivos:\n");
714     ArrayList<Dispositivo> dispositivosClientes = clienteActual.getModem().verificarDispositivos(Dispositivo.getDispositivosTotales(),
715                                                                                             clienteActual.getModem());
716     dispositivosClientes.stream().forEach(System.out::println);
717     System.out.print(s:"Indica el número del dispositivo que deseas remover: ");
718     opc=sc.nextInt();
719
720     if(opc<=dispositivosClientes.size()){
721         System.out.println(Dispositivo.desconectarDispositivo(clienteActual, dispositivosClientes.get(opc-1)));
722         Vcontrol1=false;
723     }else{
724         System.out.print(s:"Opcion incorrecta.");
725         Vcontrol1=false;
726     }

```

Así se vería en la consola:

```

Digita por favor el número de la opcion que deseas: 4
Ingresa tu Id:
333
Ingresa tu nombre:
alejandra
Ingresa el nombre de tu proveedor:
movistar
Ping: 916 ms
Red saturada
Te recomendamos remover dispositivos
Dispositivos:

1. Dispositivo: Celular
Dirección IP: 44.67.186.26
Generación: 2G
2. Dispositivo: Computador
Dirección IP: 44.67.186.26
Generación: 2G
3. Dispositivo: Celular
Dirección IP: 44.67.186.26
Generación: 4G
4. Dispositivo: Computador
Dirección IP: 44.67.186.26
Generación: 3G
5. Dispositivo: Computador
Dirección IP: 44.67.186.26
Generación: 3G
Indica el número del dispositivo que deseas remover: _

```

(Ejemplo específico de test con la red saturada).

– Segundo caso: Si el resultado indica que la generación del módem del clienteActual es incompatible con la antena a la que está conectado (mediante el método contains() de la clase String), se notifica al usuario sobre el problema y se le ofrece una recomendación para cambiar a una antena compatible.

```
727 }else if (resultado.contains(s:"incompatible")) {
728     System.out.print("El test ha detectado problemas con tu red." + "\n");
729     System.out.println(resultado);
730     System.out.print("Lo anterior quiere decir que la generación de tu router es incompatible con la antena a la que estás conectado." + "\n");
731     System.out.println(x:"Esta antena es compatible y accesible: ");
732     System.out.println(clienteActual.getMódem().getAntenaAsociada().rastrearGeneracionCompatible(antenasSede, clienteActual.getMódem()));
733
734     System.out.println("Deseas cambiar a la nueva antena:" + "\n1.Si" + "\n2.No");
```

Para la recomendación se invoca al método rastrearGeneracionCompatible de la clase Antena, que usa como parámetros al arreglo antenasSede y el módem del clienteActual. Este, el **cuarto método** de la funcionalidad, identifica una antena adecuada y devuelve el objeto Antena que cumple con las características necesarias para el cambio.

```
732 System.out.println(clienteActual.getMódem().getAntenaAsociada().rastrearGeneracionCompatible(antenasSede, clienteActual.getMódem()));
```

El usuario puede entonces decidir si realizar el cambio de antena. Así se ve por consola:

```

Digita por favor el número de la opción que deseas: 4
Ingresa tu Id:
2777
Ingresa tu nombre:
gen
Ingresa el nombre de tu proveedor:
movistar
El test ha detectado problemas con tu red.
Motivo: Generación incompatible
Lo anterior quiere decir que la generación de tu router es incompatible con la antena a la que estás conectado.
Esta antena es compatible y accesible:
Identificador: 1
Sede: C Ubicación: (70.0,5.0) Generación: 5
Deseas cambiar a la nueva antena:
1.Si
2.No

```

(Ejemplo específico de test con incompatibilidad).

Si el usuario opta por realizar el cambio de antena, se verifica si la antenaNueva es compatible (obtenida con rastrearGeneracionCompatible) pertenece al mismo proveedor que el cliente. Si es así, el cambio se realiza sin costo adicional, pero, si la antenaNueva es de otro proveedor, se informa al usuario sobre un costo de roaming y se solicita el pago correspondiente. Una vez realizado el pago, el cambio se completa, y la funcionalidad finaliza.

```

Deseas cambiar a la nueva antena:
1.Si
2.No
1
Esta antena pertenece a otro proveedor, por tanto el roaming tiene un costo de 100000
Ingresa el total a pagar: 100000
Te han quedado 0 pesos.
Tu proceso ha finalizado con éxito.

```

(Ejemplo específico de cambio Antena con cambio de proveedor).

– Tercer caso: Si el resultado indica que el clienteActual está fuera de la zona de cobertura de su antena asociada, es decir, que es inaccesible debido a problemas de cobertura, se sigue un procedimiento similar al del segundo caso: se notifica al usuario sobre el problema y se le ofrece una recomendación para cambiar a una antena accesible y compatible.

```

778 }else if(resultado.contains(s:"Inaccesible")){
779     System.out.print("El test ha detectado problemas con tu red." + "\n");
780     System.out.println(resultado);
781     System.out.print("Lo anterior quiere decir que te encuentras fuera de la zona de cobertura de la antena que tienes asociada." + "\n");
782     System.out.println(x:"Esta antena es compatible y accesible: ");
783     System.out.println(clienteActual.getModem().getAntenaAsociada().rastrearGeneracionCompatible(antenasSede, clienteActual.getModem()));

```

Para la recomendación, vuelve a invocarse el método `rastrearGeneracionCompatible` de la clase `Antena`, que usa como parámetros al arreglo `antenasSede` y el módem del `clienteActual`. Este, el mismo [cuarto método](#) de la funcionalidad, identifica una antena adecuada y devuelve el objeto `Antena` que cumple con las características necesarias para el cambio.

```

732 System.out.println(clienteActual.getModem().getAntenaAsociada().rastrearGeneracionCompatible(antenasSede, clienteActual.getModem()));

```

El usuario puede entonces decidir si realizar el cambio de antena. Así se ve por consola:

```

El test ha detectado problemas con tu red.
Motivo: Inaccesible a la zona de cobertura.
Lo anterior quiere decir que te encuentras fuera de la zona de cobertura de la antena que tienes asociada.
Esta antena es compatible y accesible:
Identificador: 2
Sede: A Ubicación: (5.0,25.0) Generación: 4
Deseas cambiar a la nueva antena:
1.Si
2.No

```

(Ejemplo específico de test con inaccesibilidad).

Si el usuario opta por realizar el cambio de antena, se comparan los proveedores y según eso, el cambio podría ser gratuito o no. Una vez realizado el pago, en caso de requerirse, el cambio se completa, tal como se explicó en el [segundo caso](#).

Finalmente, si el resultado del test no presenta problemas, se informa al usuario que su red está en buen estado y no requiere ajustes. En este caso, el programa finaliza sin realizar cambios adicionales.

```

Digita por favor el número de la opción que deseas: 4
Ingresa tu Id:
222
Ingresa tu nombre:
marcela
Ingresa el nombre de tu proveedor:
movistar
El test realizado indica que tu red no se encuentra saturada y la antena a la que estás conectado es la ideal entre todas las alternativas posibles.
Ping: 91 ms
Tu proceso ha finalizado con éxito.

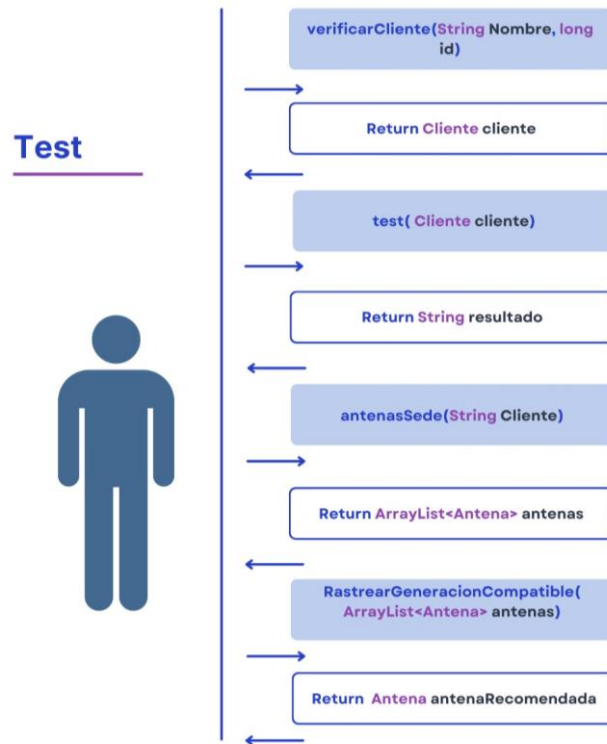
```

Observación 1: En caso de que los datos ingresados por el usuario no coincidan con ningún cliente o proveedor en la base de datos, se solicita al usuario que ingrese nuevamente su información. Este proceso se repite hasta que los datos sean válidos.

Observación 2: Este flujo asegura que los usuarios puedan identificar y resolver problemas en su red de manera eficiente, optimizando su conexión y mejorando su experiencia general con el sistema.

Observación 3: Para verificar si se ejecutó correctamente esta funcionalidad, se puede volver a seleccionar esta opción en el menú inicial (opción #4) que sale automáticamente al terminar el proceso, y confirmar que sí se haya eliminado la antena seleccionada.

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad.



5.5. Funcionalidad 5: Reporte de fallas en el servidor.

El propósito de esta funcionalidad es verificar el estado de los servidores de un proveedor de Internet y alertar sobre posibles fallas. Solicita el nombre del proveedor, valida su existencia y permite al usuario seleccionar una sede específica. Luego analiza el desempeño del servidor en esa sede, clasificando los clientes por plan ("BASIC", "STANDARD", "PREMIUM") y evaluando las intensidades de flujo reales versus las adecuadas. Si encuentra anomalías, genera un reporte con recomendaciones, incluyendo distancias óptimas entre clientes y servidor, y alerta si el servidor está saturado. Todo esto busca garantizar un servicio eficiente y de calidad, que solucione problemas de manera oportuna. Aquí intervienen objetos tipo Servidor, ProveedorInternet, Router y Cliente.

A continuación, se detalla el funcionamiento del código paso a paso:

El código inicia solicitando al usuario el nombre (string) de su proveedor / compañía prestadora de servicios de Internet, que se captura con la ayuda del objeto scanner (con apuntador sc), y que se almacena en una variable llamada nombre.

```

Ingresa el nombre de tu compañía:
tigo
  
```

(Ejemplo específico de una entrada de datos).

Se entra en un bucle while controlado por una variable Vcontrol7, inicializada en false y que verifica que el arreglo de servidores retornado por el método verificarAdmin de la clase Servidor no esté vacío, que cuente con servidores y que estos puedan obtenerse.

Se crea un objeto Servidor aleatorio con el método estático adminServidor para realizar estas verificaciones. Al validarse, se utiliza a verificarAdmin, el **primer método** de la

funcionalidad, perteneciente a la clase Servidor. Este método recibe al arreglo estático de proveedores totales (obtenido desde la clase ProveedorInternet) y al nombre (variable ingresada) como parámetros, y su función es devolver la lista de servidores asociados a ese proveedor, que se guardan en una nueva variable llamada servidoresProveedor (cuando el bucle se rompe si la lista de servidores no está vacía).

```
907 ArrayList<Servidor> servidoresProveedor = Servidor.adminServidor().verificarAdmin(ProveedorInternet.getProveedoresTotales(), nombre);
```

Se muestra por consola la lista completa de las localidades donde se encuentran los servidores del proveedor escogido (servidoresProveedor) y, para que el usuario pueda seleccionar el servidor del cual desea obtener el reporte correspondiente, ingresa el número de la sede respectiva que desea revisar.

```
La compañía de Internet tigo tiene servidores en las siguientes sedes:
1. A
2. B
3. C
4. D
Ingresa la sede que deseas revisar (Indica el número de la opción):
```

(Ejemplo específico de reporte de fallas con proveedor Tigo).

El número de sede es validado en un bucle while controlado por Vcontrol8. Si el número ingresado está dentro del rango válido (entre 1 y el tamaño de la lista de servidores), se obtiene el servidor correspondiente de la lista y se almacena en la variable servidorLocalidad. Una vez se obtenga ese servidor, se ejecutaría el [segundo método](#) de esta funcionalidad.

```
932 ArrayList<Cliente> clientesBasic = Cliente.buscarCliente1(servidorLocalidad.getSede(), nombrePlan:"BASIC",
933     servidorLocalidad.getProveedor());
934 ArrayList<Cliente> clientesStandard = Cliente.buscarCliente1(servidorLocalidad.getSede(),
935     nombrePlan:"STANDARD", servidorLocalidad.getProveedor());
936 ArrayList<Cliente> clientesPremium = Cliente.buscarCliente1(servidorLocalidad.getSede(), nombrePlan:"PREMIUM",
937     servidorLocalidad.getProveedor());
```

Se llama tres veces al método buscarCliente1 de la clase Cliente (porque son 3 planes). Este método filtra los clientes del servidor según el plan al que pertenecen: "BASIC", "STANDARD" o "PREMIUM". Recibe como parámetros la sede del servidor (string), el nombre del plan (string) y el proveedor asociado (ProveedorInternet), devolviendo un arreglo de clientes por cada plan. Los resultados se almacenan en tres listas: clientesBasic, clientesStandard y clientesPremium.

Después, se invoca al [tercer método](#) de esta funcionalidad: intensidadFlujoClientes de la clase Router, que calcula dos tipos de intensidades de flujo para cada plan: las reales y las adecuadas (estimadas). Este método recibe como parámetros la lista de clientes correspondiente (obtenidas en el paso anterior), el servidor asociado y un booleano que determina si se calculan intensidades reales (true) o adecuadas (false). Los resultados de estos cálculos se almacenan en seis listas, tres para las intensidades reales y tres para las intensidades adecuadas, correspondientes a cada plan.

```
941 // SE CALCULA LAS INTENSIDADES REALES DE LOS CLIENTES, PERO ESTO ES CLASIFICADO POR PLAN
942 ArrayList<Integer> intensidadBasic = Router.adminRouter().intensidadFlujoClientes(clientesBasic,servidorLocalidad, Reales:true);
943 ArrayList<Integer> intensidadStandard = Router.adminRouter().intensidadFlujoClientes(clientesStandard, servidorLocalidad, Reales:true);
944 ArrayList<Integer> intensidadPremium = Router.adminRouter().intensidadFlujoClientes(clientesPremium,servidorLocalidad, Reales:true);
945
946 // SE CALCULA LAS INTENSIDADES QUE DEBERIAN LLEGARLE A LOS CLIENTES, PERO ESTO ES CLASIFICADO POR PLAN
947 ArrayList<Integer> intensidadB = Router.adminRouter().intensidadFlujoClientes(clientesBasic,servidorLocalidad, Reales:false);
948 ArrayList<Integer> intensidadS = Router.adminRouter().intensidadFlujoClientes(clientesStandard,servidorLocalidad, Reales:false);
949 ArrayList<Integer> intensidadP = Router.adminRouter().intensidadFlujoClientes(clientesPremium,servidorLocalidad, Reales:false);
```

Con las intensidades obtenidas, se calcula un promedio para cada lista utilizando stream() y mapToInt(Integer::intValue).sum(). Se obtiene un promedio para las intensidades reales y

uno para las adecuadas. Se comparan ambos promedios.

Si algún promedio real es menor que el adecuado, se determina que hay problemas en el servidor y se hace un reporte.

En este caso, se calcula el promedio de distancias óptimas entre los clientes y el servidor utilizando el método `distanciasOptimas` de la clase `Servidor`. Este método toma como parámetros la lista de clientes y las intensidades adecuadas (ambos separados por plan), devolviendo un arreglo de distancias óptimas. Los promedios de estas distancias se calculan de forma similar al paso anterior.

Se mostraría en consola otro reporte con información detallada sobre las diferencias entre las intensidades reales y las adecuadas, clasificado por plan, así como las distancias óptimas recomendadas para cada plan.

```
1
De acuerdo con el análisis realizado al servidor de la sede A se hallaron errores en el servidor;
a continuación se presentan las recomendaciones y el reporte.
REPORTE
Intensidades:
Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.
Para ello, se clasificaron las intensidades por planes y de esta manera identificar el plan más afectado.
PLAN BASIC
Intensidad de Flujo Promedio Neta: 524
Intensidad de Flujo Promedio Adecuado: 534
PLAN STANDARD
Intensidad de Flujo Promedio Neta: 624
Intensidad de Flujo Promedio Adecuada: 637
PLAN PREMIUM
Intensidad de Flujo Promedio Neta: 497
Intensidad de Flujo Promedio Adecuada: 506
Las intensidades dependen en gran medida de las distancias entre el router de los clientes y el servidor,
por ello se recomienda cambiar la ubicación de este último.
A continuación, se presentan las distancias promedios recomendadas a las cuales debe estar el servidor
para que proporcione la intensidad de flujo adecuada en cada plan.
DISTANCIAS ÓPTIMAS
Distancia Promedio-Plan Basic: 11 metros.
Distancia Promedio-Plan Standard: 7 metros.
Distancia Promedio-Plan Premium: 10 metros.
Finalmente, las intensidades de flujo reales están afectadas por el porcentaje de eficiencia del servidor,
esto indica que el servidor está consumiendo una cantidad significativa
del flujo de red inicial que le proporciona el proveedor. Por ende, se recomienda cambiarlo o en su defecto,
implementar la solución de las distancias, expuesta anteriormente.
```

(Ejemplo específico de reporte de fallas con proveedor tigo en sede A).

En caso de que los servidores alcancen su límite de routers conectados (cuando se analice el índice de saturación del servidor), se indica por consola una alerta recomendando ampliar su capacidad. Se vería algo así:

```
System.out.print("ALERTA: El servidor ya se encuentra saturado, es decir, ya tiene la
cantidad de routers límite conectados. Se recomienda ampliar su capacidad.");
```

(Caso hipotético en el cual el servidor se encuentra lleno).

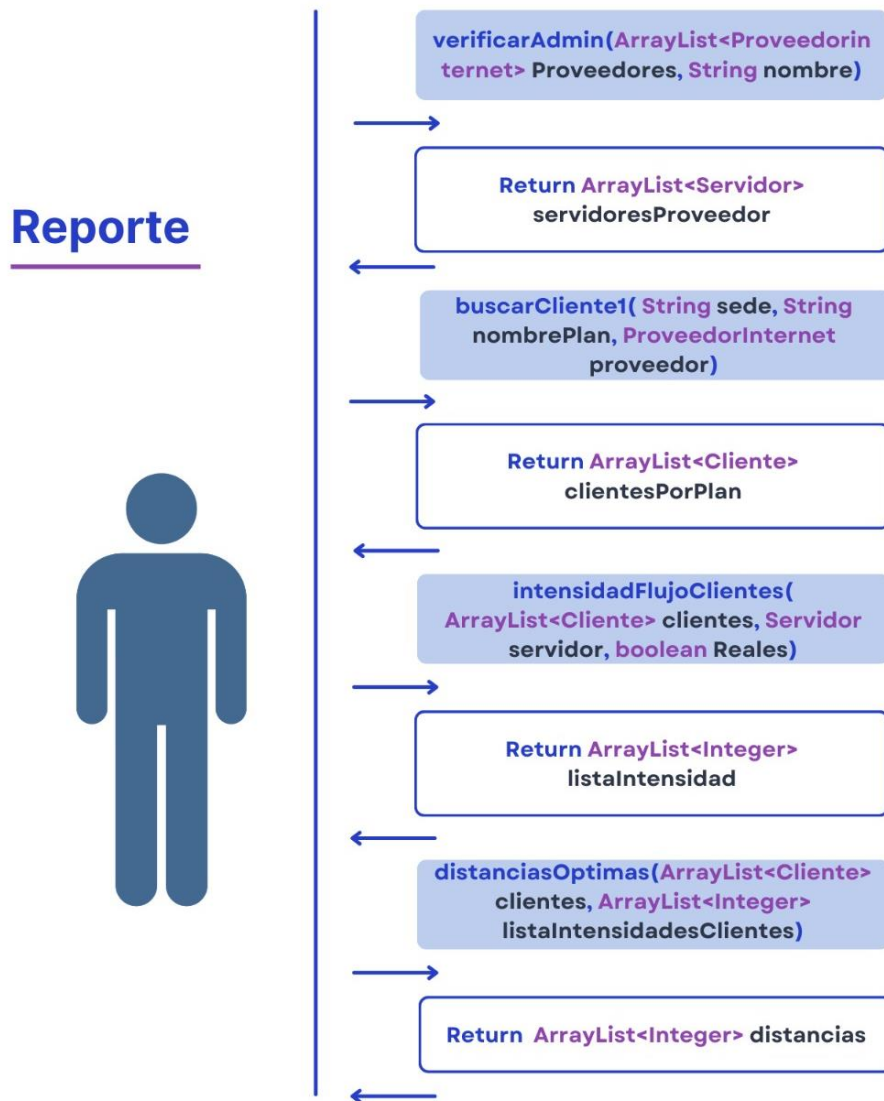
Del mismo modo, si no se detectan anomalías, se informa que el servidor funciona correctamente.

```
} else { //NO SE ENCONTRARON ANOMALÍAS
System.out.print("El servidor funciona correctamente y proporciona las intensidades de flujo
adecuadas.");
```

(Caso hipotético en el cual no se encuentran anomalías).

Finalmente, si el nombre ingresado inicialmente no corresponde a un proveedor válido, se solicita al usuario re ingresar el nombre de su compañía, repitiendo el proceso hasta que se proporcione un dato válido.

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad.



Observación: es importante mencionar que al final del código que describe las funcionalidades, en casos del 1 al 5, en el método main, existe un sexto adicional en el que se puede salir del programa, muestra un mensaje de despedida y ejecuta el método serializar() de la clase Serializador para guardar los datos.

6. Manual de usuario.

Bienvenido al sistema de gestión de proveedores de internet. Este sistema tiene como objetivo facilitar la administración de clientes, planes de servicio, y otros aspectos relacionados a la cobertura y el rendimiento de la red.

En este manual, aprenderás cómo interactuar con el sistema a través de las funcionalidades disponibles.

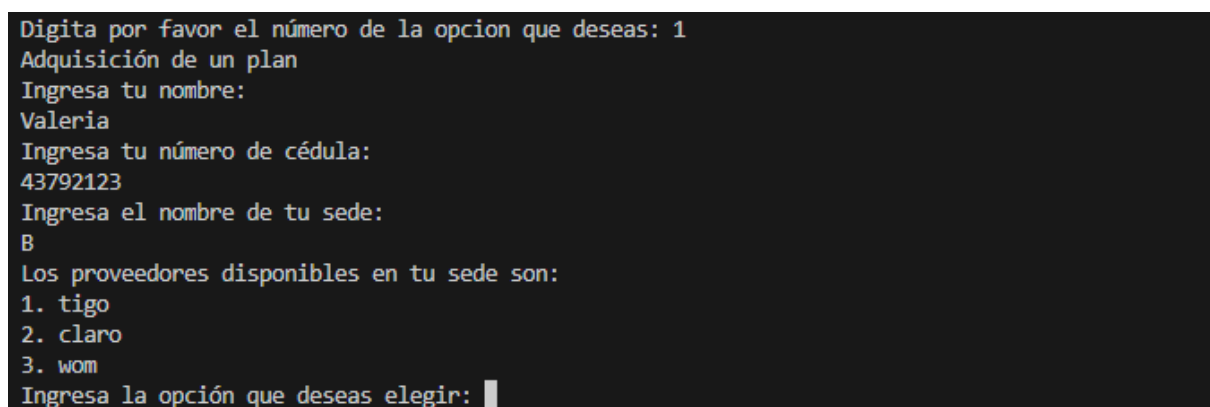
Al iniciar la aplicación, el sistema presentará un menú principal con varias opciones numeradas, que guiarán al usuario a través de cada funcionalidad.

Para comenzar, el usuario digita el número de la opción que desea utilizar. Se recuerda que siempre, seleccionar la opción 6, finalizará y guardará los cambios realizados.



6.1. Seleccionar la opción 1.

Automáticamente, al seleccionar la opción, se pedirán los siguientes datos en consola:



(Ejemplo específico de entrada de datos en la funcionalidad 1).

Con el nombre del usuario, su número de cédula, el nombre de la sede en que se encuentre (las opciones son A, B, C o D), se desplegarán los proveedores disponibles de acuerdo a la localidad seleccionada. Estas son las opciones:

- Sede A: los proveedores disponibles son Claro, Tigo, Movistar.

- Sede B: los proveedores disponibles son Claro, Tigo, Wom.
- Sede C: los proveedores disponibles son Tigo, Movistar.
- Sede D: los proveedores disponibles son Tigo, Movistar, Wom.

Se ingresa el número correspondiente al proveedor elegido (entre las opciones de su sede) y se mostrarán en pantalla los tipos de planes ofrecidos al público, que siempre se dividen en tres opciones: Basic, Standard y Premium. El precio claramente varía con el tipo de plan elegido y el proveedor que lo ofrece, las opciones disponibles son:

PROVEEDOR	Tipo de plan	Megas de subida	Megas de bajada	Precio (COP)
TIGO	Basic	10	30	\$ 30.000,00
	Standard	30	50	\$ 55.000,00
	Premium	80	100	\$ 87.000,00
CLARO	Basic	15	35	\$ 45.000,00
	Standard	25	45	\$ 76.500,00
	Premium	83	115	\$ 150.000,00
MOVISTAR	Basic	20	30	\$ 50.000,00
	Standard	28	70	\$ 60.000,00
	Premium	70	97	\$ 95.000,00
WOM	Basic	15	25	\$ 25.000,00
	Standard	23	34	\$ 46.000,00
	Premium	50	70	\$ 80.000,00

En pantalla se verá de la siguiente manera:

```

Ingresa la opción que deseas elegir: 1
Los planes disponibles del proveedor son:
BASIC
El plan basic ofrece:
Megas de subida: 10
Megas de bajada: 30
El precio del plan: 30000
STANDARD
El plan standard ofrece:
Megas de subida: 30
Megas de bajada: 50
El precio del plan: 55000
PREMIUM
El plan premium ofrece:
Megas de subida: 80
Megas de bajada: 100
El precio del plan: 87000
Ingresa el nombre del plan que deseas adquirir:

```

(Ejemplo específico plan tigo, sede B).

El siguiente paso será ingresar su ubicación (en coordenadas), se especificarán los límites de su zona, sin embargo, los límites en general son:

ZONA	Coordenadas eje X		Coordenadas eje Y	
	Límite inferior	Límite superior	Límite inferior	Límite superior
A	0	40	0	40
B	0	40	60	100
C	50	80	0	30
D	60	100	40	80

Así se ve por consola:

```

Digita las coordenadas de tu ubicación, estas tienen un límite de acuerdo con tu sede.
Coordenadas en el eje X:
Límite Inferior: 0
Límite Superior: 40
Coordenadas en el eje Y:
Límite Inferior: 60
Límite Superior: 100
Ingresa la coordenada X:

```

Luego, se podrá visualizar la factura, donde se incluirá la información de pagos atrasados (en caso de existir) y se deberá cancelar el valor total de la membresía.

```

Ingresa la coordenada X: 20
Ingresa la coordenada Y: 80
Tu plan ha sido configurado correctamente. A continuación puedes visualizar tu factura.
Factura
Usuario: valeria
Cédula: 43792123
IP: 103.43.167.54
Mes de Activación: ENERO
Número de pagos atrasados: 0
Precio total a pagar: 55000
Realiza el pago de tu membresía a continuación (Ingresa el valor a pagar):

```

En este momento, el sistema retornará al menú principal donde, se deberá seleccionar la opción 6, para que tus datos sean guardados correctamente.

Al no hacerlo, no se guardará el registro de los datos ingresados hasta el momento y no serían útiles para usarlos en siguientes funcionalidades. Se debería volver a correr el programa e ingresar de nuevo la información.

```

Menú Inicial

1. Registro y Adquisición de Plan.
2. Visualizar los dispositivos conectados.
3. Mejora tu plan.
4. Test.
5. Reporte de fallas en el servidor.
6. Salir.

Digita por favor el número de la opción que deseas: 3
Ingresa tu Id:
43792123
Ingresa tu nombre:
Valeria
Ingresa el nombre de tu proveedor:
tigo
Datos incorrectos, introdúcelos de nuevo.
Ingresa su Id:

```

6.2. Seleccionar la opción 2.

Automáticamente, al seleccionar la opción, se pedirán los siguientes datos en consola:

```
Digita por favor el número de la opción que desees: 2
Ingresa tu Id:
1234
Ingresa el nombre de tu proveedor:
tigo
```

Una vez diligenciado estos campos, se verificará si no tienes pagos pendientes. Existen dos caminos desde aquí, dependiendo si se cuentan con deudas o no.

6.2.1 Pagos al día.

Se desplegará el siguiente menú cuando el usuario tenga sus pagos al día. Existen 5 opciones diferentes, y la última nos regresa al menú inicial de opciones de funcionalidades.

```
¡Bienvenido!
Aquí podrás visualizar los dispositivos conectados a tu router.
Digita por favor la opción que desees:
1. Visualizar dispositivos de 3G.
2. Visualizar dispositivos de 4G.
3. Visualizar dispositivos de 5G.
4. Visualizar todos los dispositivos conectados.
5. Regresar al menú inicial.
```

Las demás opciones 4 mostrarán, de manera muy clara, exactamente lo descrito, con su respectiva información detallada, como se ve a continuación:

1. Dispositivos 3G:

```
1
Dispositivos de 3G conectados:
Id: 4
Dispositivo: Computador
Dirección IP: 85.5.162.60
Generación: 3G
Id: 5
Dispositivo: Computador
Dirección IP: 85.5.162.60
Generación: 3G
```

2. Dispositivos 4G:

```
2
Dispositivos de 4G conectados:
Id: 3
Dispositivo: Celular
Dirección IP: 85.5.162.60
Generación: 4G
```

3. Dispositivos 5G:

```
3
Dispositivos de 5G conectados:
Id: 1
Dispositivo: Celular
Dirección IP: 85.5.162.60
Generación: 5G
Id: 2
Dispositivo: Computador
Dirección IP: 85.5.162.60
Generación: 5G
```

4. Todos los dispositivos conectados:

```
4
Lista de dispositivos conectados a tu Router:
Id: 1
Dispositivo: Celular
Dirección IP: 85.5.162.60
Generación: 5G
Id: 2
Dispositivo: Computador
Dirección IP: 85.5.162.60
Generación: 5G
Id: 3
Dispositivo: Celular
Dirección IP: 85.5.162.60
Generación: 4G
Id: 4
Dispositivo: Computador
Dirección IP: 85.5.162.60
Generación: 3G
Id: 5
Dispositivo: Computador
Dirección IP: 85.5.162.60
Generación: 3G
```

6.2.2 Pagos pendientes:

Esta opción tiene un sistema para verificar los pagos pendientes de las personas, el requisito para pasar el filtro es no tener más de 2 pagos atrasados, por lo que se solicitará un abono a la deuda o pagar la totalidad de la deuda para poder continuar, de lo contrario está la opción de regresar al menú inicial.

```
¡Bienvenido!
Aquí podrás visualizar los dispositivos conectados a tu router.

Tienes 3 pagos atrasados, para acceder a esta función debes tener 2 o menos pagos atrasados.
Tu factura actualmente está por un valor de $180000, recuerda que el valor mensual de tu plan es de $60000
Tenemos las siguientes opciones de pago:
1. Abonar a la deuda.
2. Pagar la totalidad de la deuda.
3. Regresar al menú inicial.

Ingresa el número de la opción que desees: _
```

Si se escoge la opción 1 de abonar a la deuda se debe abonar el monto necesario para quedar con máximo 2 pagos atrasados, esto nos permitirá continuar con el mismo proceso visto anteriormente en **6.2.1**:

```

Ingresa el número de la opción que deseas: 1
Digita por favor la cantidad a abonar:
60000
Digita por favor la opción que deseas:
1. Visualizar dispositivos de 3G.
2. Visualizar dispositivos de 4G.
3. Visualizar dispositivos de 5G.
4. Visualizar todos los dispositivos conectados.
5. Regresar al menú inicial.

```

Si el abono a la deuda es inferior a lo necesario para quedar con máximo 2 pagos atrasados, el sistema dirá que el monto no es suficiente y nos devolverá automáticamente al menú inicial.

```

Ingresa el número de la opción que deseas: 1
Digita por favor la cantidad a abonar:
50000
La cantidad ingresada no es suficiente para acceder a esta función.

```

De manera similar si elegimos la opción 2 de pagar la totalidad de la deuda se nos habilitará las opciones del apartado 6.2.1.

```

Ingresa el número de la opción que deseas: 2
Digita por favor la cantidad total a pagar:
180000
Digita por favor la opción que deseas:
1. Visualizar dispositivos de 3G.
2. Visualizar dispositivos de 4G.
3. Visualizar dispositivos de 5G.
4. Visualizar todos los dispositivos conectados.
5. Regresar al menú inicial.

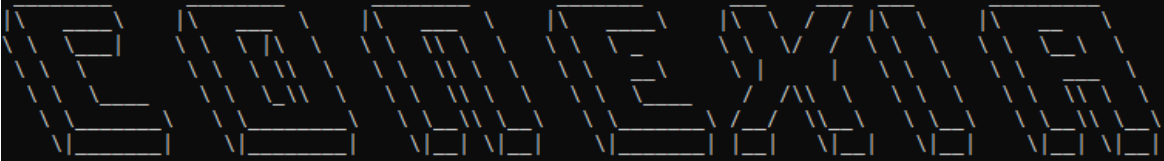
```

Y por último como se ha dicho anteriormente, si el usuario no cuenta con la capacidad de pagar la totalidad o abonar lo suficiente, tendrá la opción 3 para regresar al menú inicial e indagar el resto de opciones.

```

Ingresa el número de la opción que deseas: 3

```



```

Menú Inicial

1. Registro y Adquisición de Plan.
2. Visualizar los dispositivos conectados.
3. Mejora tu plan.
4. Test.
5. Reporte de fallas en el servidor.
6. Salir.

Digita por favor el número de la opción que deseas:

```

6.3. Seleccionar la opción 3.

Automáticamente, al seleccionar la opción, se pedirán los siguientes datos en consola:

```

Digita por favor el número de la opcion que deseas: 3
Ingresa tu Id:
1789
Ingresa tu nombre:
Margarita
Ingresa el nombre de tu proveedor:
movistar

```

Después de ingresarlos correctamente, se mostrará una recomendación de mejora para el plan del usuario, quien deberá leer cuidadosamente si quiere cambiar su plan actual por el plan que le recomienda el sistema.

```

Al realizar las verificaciones encontramos que este nuevo servicio puede ser mejor para ti.
Te presentamos la siguiente recomendación:
Intensidad de flujo: 521
Proveedor: movistar
Plan STANDARD
Megas de subida: 28
Megas de bajada: 70
Precio: 60000
Tu plan actual tiene las siguientes características:
Intensidad de flujo: 503
Proveedor: tigo
Plan STANDARD
Megas de subida: 30
Megas de bajada: 50
Precio: 55000
¿Deseas cambiar tu plan por la recomendación generada?
1. Sí
2. No

```

El usuario puede cambiar el plan si así lo desea, pero sólo si tiene los pagos al día. En caso de no ser así, se le ofrecerá la opción de pagar la totalidad de la deuda.

```

¿Deseas cambiar tu plan por la recomendación generada?
1. Sí
2. No
1

Tienes 4 pagos atrasados, para continuar debes saldar la deuda.
Tu factura actualmente está por un valor de $240000, elige una opción:
1. Pagar la totalidad de la deuda.
2. Salir.

```

Al eliminar la deuda por completo, el plan será actualizado con éxito.

```

1
Digite por favor la cantidad total a pagar:
240000
Tu plan ha sido actualizado con éxito.

```

En el caso de no tener deudas, el plan se cambiará automáticamente.

En este momento, el sistema retornará al menú principal donde, se deberá seleccionar la opción 6, para que tus datos sean guardados correctamente.

6.4. Seleccionar la opción 4.

Automáticamente, al seleccionar la opción, se pedirán los siguientes datos en consola:

```
Ingresar tu Id:
0
Ingresar tu nombre:
cob
Ingresar el nombre de tu proveedor:
tigo
```

Se hará un test en la red del usuario y se le notificará en caso de encontrar problemas, así mismo, se le ofrecerá una posible solución. Existen 3 tipos de problemas:

- Si la red está saturada, la solución recomendada será remover uno o varios de los dispositivos que hay conectados, sólo se debería ingresar el nombre del dispositivo.

```

Digita por favor el número de la opción que deseas: 4
Ingresar tu Id:
333
Ingresar tu nombre:
alejandra
Ingresar el nombre de tu proveedor:
movistar
Ping: 916 ms
Red saturada
Te recomendamos remover dispositivos
Dispositivos:

1. Dispositivo: Celular
Dirección IP: 44.67.186.26
Generación: 2G
2. Dispositivo: Computador
Dirección IP: 44.67.186.26
Generación: 2G
3. Dispositivo: Celular
Dirección IP: 44.67.186.26
Generación: 4G
4. Dispositivo: Computador
Dirección IP: 44.67.186.26
Generación: 3G
5. Dispositivo: Computador
Dirección IP: 44.67.186.26
Generación: 3G
Indica el número del dispositivo que deseas remover: _
```

- Si hay incompatibilidad entre el módem del cliente y la antena a la que se conecta, la solución recomendada para este problema es cambiar de antena a una que sea compatible con la generación.


```

Digita por favor el número de la opción que deseas: 4
Ingresa tu Id:
2777
Ingresa tu nombre:
gen
Ingresa el nombre de tu proveedor:
movistar
El test ha detectado problemas con tu red.
Motivo: Generación incompatible
Lo anterior quiere decir que la generación de tu router es incompatible con la antena a la que estás conectado.
Esta antena es compatible y accesible:
Identificador: 1
Sede: C Ubicación: (70.0,5.0) Generación: 5
Deseas cambiar a la nueva antena:
1.Si
2.No

```

- Si hay inaccesibilidad debido a problemas de cobertura de la antena, la solución recomendada es cambiar la conexión a la antena recomendada para cada caso.

```

El test ha detectado problemas con tu red.
Motivo: Inaccesible a la zona de cobertura.
Lo anterior quiere decir que te encuentras fuera de la zona de cobertura de la antena que tienes asociada.
Esta antena es compatible y accesible:
Identificador: 2
Sede: A Ubicación: (5.0,25.0) Generación: 4
Deseas cambiar a la nueva antena:
1.Si
2.No

```

En caso de que el cliente no tenga problemas o que ya los haya solucionado y quiera verificar que sí funcionó, se le mostrará un ping que será aceptable cuando sea menor a 100 ms.

```

Digita por favor el número de la opción que deseas: 4
Ingresa tu Id:
222
Ingresa tu nombre:
marcela
Ingresa el nombre de tu proveedor:
movistar
El test realizado indica que tu red no se encuentra saturada y la antena a la que estás conectado es la ideal entre todas las alternativas posibles.
Ping: 91 ms
Tu proceso ha finalizado con éxito.

```

6.5. Seleccionar la opción 5.

Automáticamente, al seleccionar la opción, se le pedirá al usuario ingresar el nombre de su proveedor y, al digitar cualquiera de las opciones (tigo, movistar, claro, wom) en consola, se mostrarán las sedes donde aquel proveedor opera y se nos solicitará escoger de cual sede nos gustaría ver el reporte.

Cada proveedor tiene servicio en diferentes sedes y son:

- Tigo tiene servidores en las sedes A, B, C, D.
- Movistar tiene servidores en las sedes A, C, D.
- Claro tiene servidores en las sedes A, B.
- Wom tiene servidores en las sedes B, D.

Este proceso no tiene mayor complejidad y se repite de manera similar en el resto de sedes, y proveedores, donde las intensidades y distancias cambian para cada uno. Para evitar redundancia en el proceso, sólo se explicará a profundidad la opción para el proveedor Tigo (es el único en las 4 sedes).

```

Digita por favor el número de la opción que deseas: 5
Ingresa el nombre de tu compañía:
tigo
La compañía de Internet tigo tiene servidores en las siguientes sedes:
1. A
2. B
3. C
4. D
Ingresa la sede que deseas revisar (Indica el número de la opción):

```

Para cada una de las opciones, se revelará la información del rendimiento actual del plan (intensidad de flujo promedio neta) y el rendimiento óptimo (intensidad de flujo promedio adecuada), también se verá una breve explicación de porque puede variar esta intensidad, se mostrarán las distancias recomendadas que debería haber entre el servidor y el cliente, se da una recomendación final y se termina la funcionalidad.

Para la opción 1 (sede A en Tigo):

```

1
De acuerdo con el análisis realizado al servidor de la sede A se hallaron errores en el servidor;
a continuación se presentan las recomendaciones y el reporte.
REPORTE
Intensidades:
Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.
Para ello, se clasificaron las intensidades por planes y de esta manera identificar el plan más afectado.
PLAN BASIC
Intensidad de Flujo Promedio Neta: 524
Intensidad de Flujo Promedio Adecuado: 534
PLAN STANDARD
Intensidad de Flujo Promedio Neta: 624
Intensidad de Flujo Promedio Adecuada: 637
PLAN PREMIUM
Intensidad de Flujo Promedio Neta: 497
Intensidad de Flujo Promedio Adecuada: 506
Las intensidades dependen en gran medida de las distancias entre el router de los clientes y el servidor,
por ello se recomienda cambiar la ubicación de este último.
A continuación, se presentan las distancias promedios recomendadas a las cuales debe estar el servidor
para que proporcione la intensidad de flujo adecuada en cada plan.
DISTANCIAS ÓPTIMAS
Distancia Promedio-Plan Basic: 11 metros.
Distancia Promedio-Plan Standard: 7 metros.
Distancia Promedio-Plan Premium: 10 metros.
Finalmente, las intensidades de flujo reales están afectadas por el porcentaje de eficiencia del servidor,
esto indica que el servidor está consumiendo una cantidad significativa
del flujo de red inicial que le proporciona el proveedor. Por ende, se recomienda cambiarlo o en su defecto,
implementar la solución de las distancias, expuesta anteriormente.

```

Para la opción 2 (sede B en Tigo):

```

2
De acuerdo con el análisis realizado al servidor de la sede B se hallaron errores en el servidor;
a continuación se presentan las recomendaciones y el reporte.
REPORTE
Intensidades:
Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.
Para ello, se clasificaron las intensidades por planes y de esta manera identificar el plan más afectado.
PLAN BASIC
Intensidad de Flujo Promedio Neta: 535
Intensidad de Flujo Promedio Adecuado: 540
PLAN STANDARD
Intensidad de Flujo Promedio Neta: 577
Intensidad de Flujo Promedio Adecuada: 582
PLAN PREMIUM
Intensidad de Flujo Promedio Neta: 668
Intensidad de Flujo Promedio Adecuada: 674

```

```
DISTANCIAS ÓPTIMAS
Distancia Promedio-Plan Basic: 15 metros.
Distancia Promedio-Plan Standard: 6 metros.
Distancia Promedio-Plan Premium: 12 metros.
```

Para la opción 3 (sede C en Tigo):

```
3
De acuerdo con el análisis realizado al servidor de la sede C se hallaron errores en el servidor;
a continuación se presentan las recomendaciones y el reporte.
REPORTE
Intensidades:
Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.
Para ello, se clasificaron las intensidades por planes y de esta manera identificar el plan más afectado.
PLAN BASIC
Intensidad de Flujo Promedio Neta: 536
Intensidad de Flujo Promedio Adecuado: 541
PLAN STANDARD
Intensidad de Flujo Promedio Neta: 581
Intensidad de Flujo Promedio Adecuada: 586
PLAN PREMIUM
Intensidad de Flujo Promedio Neta: 779
Intensidad de Flujo Promedio Adecuada: 786
```

```
DISTANCIAS ÓPTIMAS
Distancia Promedio-Plan Basic: 14 metros.
Distancia Promedio-Plan Standard: 2 metros.
Distancia Promedio-Plan Premium: 11 metros.
```

Para la opción 4 (sede D en Tigo):

```
4
De acuerdo con el análisis realizado al servidor de la sede D se hallaron errores en el servidor;
a continuación se presentan las recomendaciones y el reporte.
REPORTE
Intensidades:
Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.
Para ello, se clasificaron las intensidades por planes y de esta manera identificar el plan más afectado.
PLAN BASIC
Intensidad de Flujo Promedio Neta: 567
Intensidad de Flujo Promedio Adecuado: 572
PLAN STANDARD
Intensidad de Flujo Promedio Neta: 573
Intensidad de Flujo Promedio Adecuada: 578
PLAN PREMIUM
Intensidad de Flujo Promedio Neta: 567
Intensidad de Flujo Promedio Adecuada: 572
```

```
DISTANCIAS ÓPTIMAS
Distancia Promedio-Plan Basic: 16 metros.
Distancia Promedio-Plan Standard: 10 metros.
Distancia Promedio-Plan Premium: 16 metros.
```

6.6. Seleccionar la opción 6.

Al seleccionar la opción 6 el programa terminará su ejecución, se despedirá, guardará los datos y cerrará hasta una próxima visita.

```
Digita por favor el número de la opcion que deseas: 6
Ha sido un gusto servirte
```

En caso de que el usuario quiera verificar el correcto funcionamiento del programa, puede seguir las siguientes recomendaciones en cada funcionalidad:

1. Funcionalidad Adquisición de un plan: puede crearse un usuario desde cero con datos aleatorios. No se necesita tener historial.
2. Funcionalidad Visualizar los dispositivos conectados: pueden usarse los datos de Id: 1234, sede: A, nombre: Andrea y proveedor: Tigo, en la consola.

Este es un buen ejemplo, los datos pertenecen a Andrea, un cliente registrado con una amplia variedad de dispositivos conectados y 7 pagos retrasados. Ella necesita un abono mínimo de 275.000 (para cumplir la condición de no tener +2 deudas) para visualizar los dispositivos, se ejecutaría el ítem **6.2.2.** para pagar y luego, verificar en **6.2.1.**

3. Funcionalidad Mejora tu plan: el usuario del ejemplo anterior es un buen ejemplo aquí también, se ingresan los datos de Id: 1234, sede: A, nombre: Andrea y proveedor: Tigo, en la consola.

El plan sería mejorado de Tigo a Movistar, seguiría siendo de tipo Standard.

4. Funcionalidad Test: existen varios usuarios ya creados que se pueden utilizar aquí:
 - En caso de saturación puede usarse a nombre: Alejandra, proveedor: Movistar, sede: A, Id: 333.
 - En caso de incompatibilidad puede usarse a nombre: Gen, proveedor: Movistar, sede: C, Id: 2777.
 - En caso de inaccesibilidad puede usarse a nombre: Cob, proveedor: Tigo, sede: A, Id: 0.
 - En caso de que el test esté bueno: se solucionan los problemas de alguno de los usuarios anteriores (Alejandra, Gen o Cob) y se comprueba aquí se haya solucionado correctamente.
5. Funcionalidad Reporte de fallas en el servidor: el único dato de entrada que se necesita aquí es el nombre del proveedor y ya los conocemos: Tigo, Claro, Wom, Movistar. No es necesario tener historial ni algún dato adicional para ejecutar.